Prague University of Economics and Business

Faculty of Informatics and Statistics

Data-Driven Analysis of Regional Housing Markets in the United States

BACHELOR THESIS

Study program: Applied informatics

Author: Herman Sachek

Bachelor thesis supervisor: prof. Ing. Petr Berka, CSc.

Prague, May 2025

Annotation

This thesis is intended solely for academic and research purposes. All data, analyses, and conclusions presented here are used exclusively for educational objectives in accordance with Fair Use principles and do not serve any commercial intent.

In Prague,
12.05.2025

....
Herman Sachek

Gratitude

I express my deep gratitude to the esteemed Mr. Prof. Ing. Petr Berka, CSc. for guiding the bachelor's thesis, patience, openness and provided valuable rules and instructions without which the writing of this bachelor's thesis and the achievement of the goals would not have been possible.

Abstract

Cílem této bakalářské práce je shromáždit data o trhu rezidenčních nemovitostí ve Spojených státech pomocí web scrapingu a analyzovat je s využitím explorační analýzy dat a technik data miningu za účelem identifikace vzorců, vztahů mezi atributy a regionálních rozdílů na trhu.

Teoretická část představuje oblast Dobývání Znalostí z Databáze a vysvětluje klíčové pojmy, včetně její definice, historického vývoje, hlavních kroků procesu a metodologie CRISP-DM. Dále se zabývá tématem web scrapingu, jeho definicí, využitím a technikami, jako je statické parsování HTML nebo extrakce dynamického obsahu.

Praktická část se řídí metodologií CRISP-DM a je aplikována na analýzu trhu nemovitostí v USA. Začíná fází porozumění podnikatelskému cíli (business understanding), včetně výběru vhodného zdroje dat a měst pro sběr dat. Data jsou získána pomocí web scrapingu a následně připravena v rámci fází Data Understanding a Data Preparation. Nakonec jsou aplikovány metody explorační analýzy dat a modelování za účelem odhalení poznatků z datové sady.

Klíčová slova

Data mining, web scrapping, datová analýza, realitní trh, USA

Abstract

The aim of this bachelor's thesis is to collect data on the residential real estate market in the United States through web scraping and to analyze it using exploratory data analysis and data mining techniques in order to identify patterns, relationships between attributes, and regional differences within the market.

The theoretical part introduces the field of Knowledge Discovery in Databases (KDD) and explains key concepts, including its definition, historical development, main process steps, and the CRISP-DM methodology. It also explores the topic of web scraping, covering its definition, applications, and the techniques used, such as static HTML parsing and dynamic content extraction.

The practical part follows the CRISP-DM framework and applies it to the analysis of the U.S. real estate market. It begins with business understanding, including the selection of a relevant data source and the choice of cities for extraction. Data is collected using web scraping and prepared through the Data Understanding and Data Preparation phases. Finally, exploratory data analysis (EDA) and modeling techniques are applied to uncover insights from the dataset.

Keywords

Data mining, web scraping, data analysis, CRISP-DM, real estate market, USA

Table of Contents

List of Figures

List of Codes

Introduction

The real estate market in the United States is one of the most complex and regionally diverse in the world. With vast differences in geography, climate, infrastructure, and urban development, each city offers its own unique housing landscape. As someone who has spent almost a year traveling extensively across various parts of the United States, I was consistently fascinated by how dramatically cities differ not only in atmosphere and lifestyle but also in the structure and pricing of their residential properties. These observations became a key motivation for choosing this topic for my bachelor's thesis.

During my stays in the U.S., I visited cities with stark contrasts—from dense metropolitan areas with advanced infrastructure to quieter, suburban towns with entirely different architectural and social dynamics. These experiences raised questions that this thesis aims to explore: How do different factors influence real estate pricing across regions? Are there measurable patterns behind these differences? Can we identify shared attributes among similar markets?

Another key motivation was the availability of large volumes of real estate data published online. Property listing websites often contain rich, detailed information about homes, including pricing, size, amenities, location, and various additional features. I was particularly interested in collecting this data, preparing it for analysis, and exploring how it could be used to better understand pricing dynamics. This led to the decision to apply web scraping techniques to extract relevant data directly from public sources.

To answer the research questions, this thesis focuses on analyzing residential real estate data across selected U.S. cities using modern data analysis techniques. By applying web scraping methods to collect real estate listings, and using exploratory data analysis (EDA) and data mining tools, the goal is to uncover patterns, correlations, and insights into how residential property markets behave across different locations.

This work is divided into two main parts: a theoretical section, which introduces key concepts from Knowledge Discovery in Databases (KDD), including the CRISP-DM methodology and web scraping techniques; and a practical section, where these concepts are applied to real-world data to gain a better understanding of regional real estate dynamics. All work is conducted strictly for academic purposes under Fair Use principles.

1 Knowledge Discovery in Databases

This chapter provides an overview of the concept of KDD, including its definition, historical development, and main steps. Special attention is given to the CRISP-DM methodology, which has become a widely adopted standard for structuring data mining projects across both academic and commercial settings.

1.1 Definition of Knowledge Discovery in Databases

Knowledge discovery in databases(KDD) is a dynamic research field that has the potential for high returns in many professional and scientific domains. Nowadays, the results of human effort are being shelved, which causes a fast and unstoppable growth in the amount of data. All these data are recorded becoming computer databases, which could be managed by computer technologies. Development of computational theories and new tools set the goal of extracting information, which could be valuable, from the soaring amount of digitalized data (Singhal & Himanshu, 2022).

In the book "Knowledge Discovery in Databases: An Overview," Knowledge database management is defined as the *nontrivial extraction of implicit, previously unknown, and potentially useful information from data* (Frawley et al., 1992).

1.2 The History of Knowledge Discovery in Databases

The history of the development of Knowledge discovery in databases takes root in the late 1980s, when G. Wiederhold and his student R. Blum developed the first program, which looked for unexpected side-effects of drugs, analyzing a dataset of historical data from approximately 50,000 patients of Stanford. The results of using this program looked very promising since it found some side effects that were unknown to the authors.

Later, in 1989, as part of the International Joint Conference on Artificial Intelligence in Detroit, a workshop dedicated to discovery in data was organized by Professor Gregory Piatetsky-Shapiro. The workshop held was named "Knowledge Discovery in Databases". In the article by Gregory Piatetsky-Shapiro, called "Knowledge Discovery in Databases: 10 years after," he explained how he came up with this name, explaining that at that time the term "data mining" already existed, but it seemed to him unattractive to use. The term "mining" didn't explain what was being mined, while "database mining" was HNC's trademark. Finally, he decided to name his workshop as "Knowledge Discovery in Databases" since it points up the aspect of "discovery" and the focus of discovery on "knowledge".

After this, in the AI and Machine Learning communities, the term KDD and the related term "data mining" became popular. Now they are often used as synonyms, although originally they have different meanings. Georgy Piatetsky-Shapiro, Fayyad, and Smyth, in their work from 1996, "From Data Mining to Knowledge Discovery in Databases," defined KDD as the broader process encompassing the entire cycle of discovering knowledge from data. It includes stages such as data selection, preprocessing, transformation, data mining, and interpretation/evaluation of results. Data Mining, in turn, is one step within the KDD process, specifically focused on the extraction of patterns and information from data using algorithms and techniques like clustering, classification, and association (Fayyad et al., 1996).

However, over time, the term “data mining” gained more popularity due to the business press, and as of November 1999, a search on www.altavista.com gave approximately 100,000 pages for the search of “data mining”, whereas a search for “knowledge discovery” gave about 18,000 results (Piatetsky-Shapiro, 2000).

One of the famous chapters in the history of knowledge discovery in databases was presented in the book Moneyball by Michael Lewis, which was published in 2003 and changed the way a lot of major baseball front offices do their business. The management of the Oakland Athletics signed undervalued and cheap baseball players using a statistical and data-driven approach. As a result, with 1/3 of the former payroll, they successfully assembled a team, which could bring them to the 2002 and 2003 playoffs (Li, n.d.).

1.3 Knowledge Database Discovery Process Steps

The process of knowledge discovery in databases includes 7 steps, which should be reviewed before analyzing datasets.

1. Cleaning of Data

   Cleaning of Data, or Data Cleaning, is the process of looking for incomplete, inaccurate, or unreasonable data and, with further elimination of omissions and possible errors, which usually includes reasonableness checks, limit checks, checks of the data to identify outliers, completeness checks, and format checks (Chapman, 2005).

   All data could be divided into primary and secondary data.

   While primary data is the data, that has been collected by researcher in the way of surveys, experiments, interviews and main reason why this data was collected is to solve the research problem what has a big advantage, since the collector, understanding the issue and the main problems of the research, can accumulate only valid and realistic data, secondary data is the data collected by government institutions, healthcare facilities or businesses as keeping the records process. (Wagh, n.d.) Secondary data nowadays is generated very fast and is being collected from various sources, which means that they can be full of dirty data, which should be cleaned before they are loaded into a warehouse in order to have only quality data. Precise results can be made only relying on cleaned data, so the consistent data is very important for decision making; that's why cleaning of secondary data is an essential process (Swapna et al., 2016).

2. Integration of Data

   Integration of Data is the process of combining data from different sources into a common source(Singhal & Himanshu, 2022). This definition is relevant to the problem of KDD. However, in the area of business data are fast gained and added to databases, which results necessity to be able to merge updated data fast from systems, which can be autonomous and have their own storing standards integration even inside one small business, so in a broader sense to the definition of data integration can be added providing unified access to data in multiple sources (Dong & Srivastava, 2022).

3. Selection of Data

   Selection of Data is the process when it is decided what data must be retrieved from data collection, with the following selection(Singhal & Himanshu, 2022). Although data can be sufficiently cleaned, analysis has to be based only on relevant data to meet the goals of the analysis. For example, analyzing bank data to understand diversity between bank accounts by the amount of money, information about the full name of holders will not be most likely useful.

4. Data Transformation

   To easily implement methods of data mining, data must be transformed from one form to another (Singhal & Himanshu, 2022). Depending on the methods and tools we may need to use different formats for the same data. For example, to be able to make an analysis using Microsoft Excel app functions and features, we need to have this data in an Excel spreadsheet, while XML format is used for storing data and further analyzing data in HTML applications.

5. Data Mining

   As already mentioned, the term “Data mining” is often used as a synonym of the term knowledge discovery in databases. However, “Data mining” is only one of the steps of KDD, which involves the use of data analysis tools to find previously unknown patterns and relationships in data sets. Among these tools are statistical models, mathematical algorithms, machine learning methods, neural networks, and decision trees.

   For data examination, a variety of parameters can be used, such as:

   - Association - one event is connected to another event. For example, buying a pen and buying paper.
   - Path analysis or sequence – one event leads to another event. For example, the birth of a child and buying diapers.

- Classification – finding new patterns, coincidences. For example, purchases of drinking coffee and plastic bags.
- Clustering – identification and visual documentation of groups of previously unknown facts. For example, brand preferences depend on geographic location.
- Forecasting – making reasonable predictions based on previously discovered patterns. For example, people who have already joined the athletic club will most likely take exercise classes.

The key difference between Data mining, as an application, and other data analysis applications is that it is based on discovery approach, what means that data mining algorithms can examine data relationships simultaneously to identify unique and frequently represented, while other data analysis applications use the verification-based approach, what means that a user creates a hypothesis and then data are used to test this hypothesis. The minus of this approach is that it is limited by the creativity of the user (Pegarkov, 2006).

6. Pattern Evaluation

A pattern is the relationship and summaries obtained through a data mining exercise. Pattern evaluation is a crucial step that determines the usefulness, accuracy, and relevance of the patterns identified during data mining. This stage involves validating the results and applying statistical or domain-specific measures to assess the significance of the patterns. Patterns are considered useful if they provide actionable insights or new knowledge. For instance, in market analysis, pattern evaluation could determine the relevance of customer purchasing trends, validating whether these trends offer any predictive value for future sales (Fayyad et al., 1996).

7. Knowledge representation

This final step in the KDD process is focused on making the discovered knowledge understandable and accessible for end-users or decision-makers. Knowledge representation organizes and visualizes the extracted patterns or insights in a meaningful way, often using visualization tools, summary reports, or dashboards. By doing so, it ensures that non-technical stakeholders can interpret the results and apply them effectively. For instance, in a business context, knowledge representation might involve summarizing customer segmentation data in a visual format, like charts or decision trees, allowing managers to make data-driven marketing decisions (Han et al., 2011).

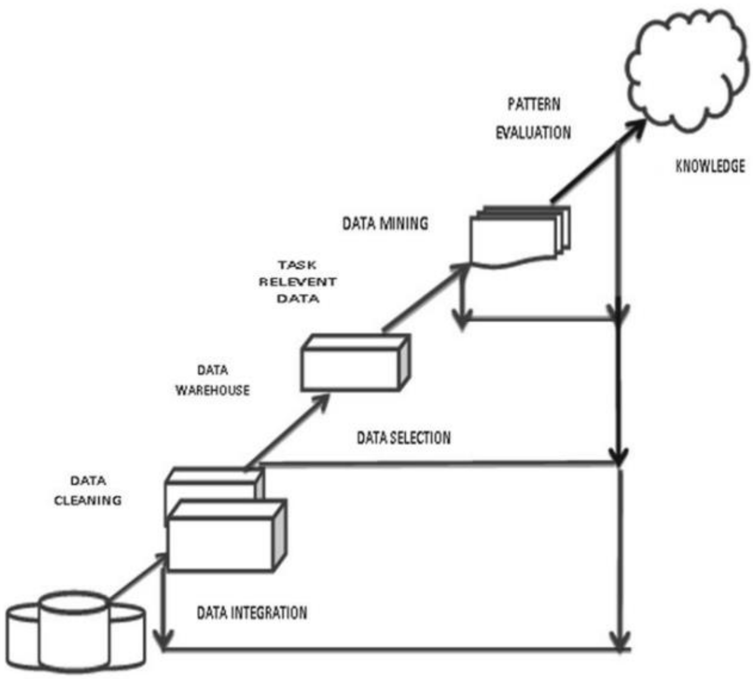

Figure 1 Knowledge Discovery Process (Singhal & Himanshu, 2022)

1.4 CRISP-DM

The Cross-Industry Standard Process for Data Mining (CRISP-DM) is a widely adopted methodology created to standardize data mining practices across various industries. Developed in the late 1990s, CRISP-DM provides a structured, business-oriented approach, making it particularly valuable for applied data mining projects (Wirth & Hipp, 2000). While Knowledge Discovery in Databases (KDD) represents the conceptual framework for extracting patterns and knowledge from data, CRISP-DM adapts these principles into a six-step process that aligns closely with business objectives.

CRISP-DM consists of six phases that parallel key steps in the KDD process but emphasize application within a business context. Another feature of CRISP-DM is its flexible and iterative approach. It can be easily seen in Figure 2, which illustrates connections between processes in CRISP-DM. Practitioners may cycle back to adjust objectives, re-select data, or refine models, similar to how Agile teams adjust project features based on stakeholder feedback.

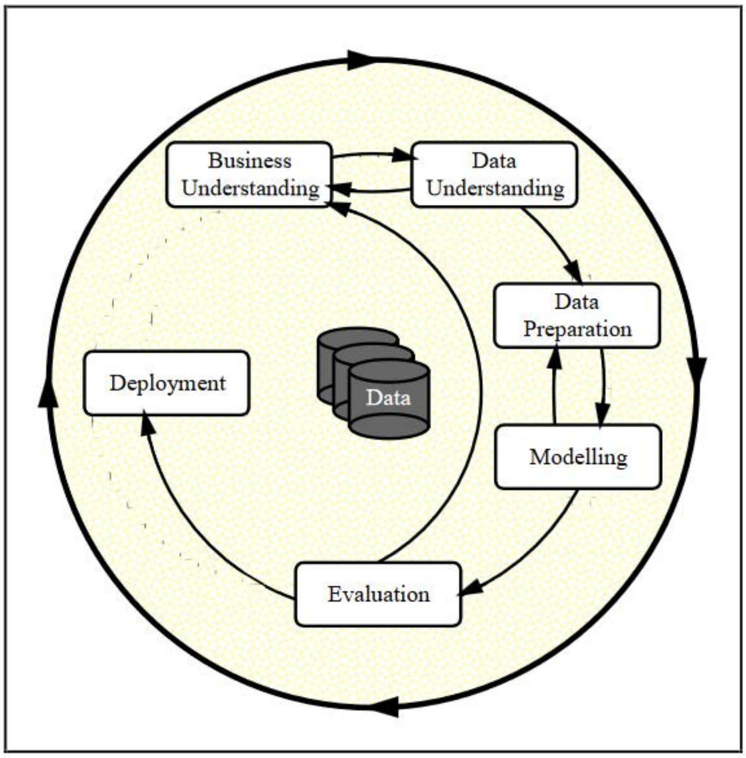

Figure 2 Phases of the Current CRISP-DM Process Model for Data Mining (Wirth & Hipp, 2000)

Business Understanding

Unlike the broader theoretical approach of KDD, CRISP-DM begins with a focus on business objectives, transforming them into concrete data mining goals. This initial phase sets the project's purpose and scope, aligning with KDD's data selection and transformation steps, but with an added emphasis on business impact (Wirth & Hipp, 2000).

Data Understanding

Similar to KDD's data preparation stages, this phase involves collecting and assessing data quality. CRISP-DM emphasizes exploratory data analysis to uncover data properties that are relevant to the business goal (Han et al., 2011).

Data Preparation

In both KDD and CRISP-DM, data preparation includes tasks like data cleaning, transformation, and selection. In CRISP-DM, these activities are guided by the initial business understanding phase to ensure that only the most relevant data is prepared for modeling (Wirth & Hipp, 2000).

Modeling

The modeling phase in CRISP-DM corresponds with the data mining step in KDD, applying algorithms to uncover patterns. Here, CRISP-DM emphasizes experimenting with different models to find the best fit for business needs, thus underscoring its practical orientation (Han et al., 2011)

Evaluation

Both KDD and CRISP-DM include an evaluation stage; however, CRISP-DM's evaluation phase emphasizes ensuring that the model meets business objectives and is ready for deployment. Statistical validation techniques, such as cross-validation and significance testing, used in KDD are also relevant here for assessing model accuracy and reliability (Wirth & Hipp, 2000).

Deployment:

While KDD concludes with knowledge representation, CRISP-DM includes a deployment phase to ensure actionable insights. This step integrates the model into the business environment, enabling data-driven decision-making and maximizing the model's business impact.

By embedding KDD's concepts within a practical, business-oriented methodology, CRISP-DM translates the theoretical principles of knowledge discovery into a repeatable cycle suited to industry applications. While KDD provides a foundation for understanding the data mining process, CRISP-DM focuses on deriving actionable business insights, bridging the gap between data analysis and operational decision-making.

2 Web Scraping

2.1 Definition of Web Scraping

Web scraping can be defined as building an automated agent to retrieve, parse, and organize data from websites. Essentially, rather than a human manually browsing, clicking, and copying content into a spreadsheet, a web scraping tool automates this process, performing it more efficiently and accurately than a person could (Broucke & Baesens, 2018).

The automated gathering of data from the internet is almost as old as the internet itself. In past terms, screen scraping, data mining, and web harvesting were widely used. Although the term web scraping is not new either, it became more popular later, and that is the most popular now by general consensus (Mitchell, 2018).

2.2 Usage of web scraping

While web browsers perform their function of displaying animations and images, laying out web pages in a manner that is visually convenient for people, in most cases, they do not provide an easy way to export the underlying data. Application Programming Interface(API) is one of the solutions that allow websites to access their data repositories, but there are still a lot of reasons why web scraping is preferred for using API: not providing API by the desired website, paid API, or not exposing all needed data, whereas a website does. All these reasons make web scraping a very useful instrument, since it is a fact that if you can see data in your web browser, you will be able to access and retrieve it using a computer program (Broucke & Baesens, 2018).

Today, web-scraping is widely used in the public and private sectors. Below are certain examples of users with real cases:

- Banks and financial institutions. The most common usage for them is to scrape competitors' information to use it to forecast in predictive models. For example, collect news articles related to the activities in the portfolio.
- HR and employee analytics. hiQ startup, which is based in San Francisco, works in this field and sells employee analyses by collecting and analyzing public profile information, for example, from LinkedIn
- Social organizations. In one of the studies, researchers built a predictive model that could identify patterns of depression and suicidal thoughts, analyzing messages scraped from blogs and social media, mostly Twitter.
- Researchers, data scientists: In another study in a paper titled "The Billion Prices Project: Using Online Prices for Measurement and Research," researchers constructed a robust daily price index for different countries, and data from the dataset they used has been scraped.

- Business owners. For example, to scrape information about varieties of the products to analyze their parameters (Broucke & Baesens, 2018).

In my research, web scraping serves as a primary method for collecting data on real estate listings and demographic information across a selection of ten U.S. cities. These cities are strategically chosen to represent distinct cultural and climate regions, allowing for a comparative analysis that captures regional variations. By automating the extraction of data from online sources, web scraping enables the collection of large, structured datasets, which can then be used to identify patterns and relationships. This approach not only facilitates efficient data collection from multiple sources but also allows for an in-depth exploration of factors such as property pricing trends, geographic preferences, and potentially unique cultural or climate-driven influences on the real estate market.

2.3 Techniques and Tools for Web Scraping

Although it was previously mentioned that a web scraper is an automated agent and often considered an alternative to APIs for obtaining data, the field of web scraping encompasses a wide range of methods and approaches. Examining the works of various authors reveals a lack of standardized classification for web scraping techniques. Some authors, for instance, include obtaining data through non-automated methods, such as “manual navigation,” where a person manually enters required data into a document, and API usage as part of web scraping techniques. This variability can be attributed to the unique data requirements of each project, as well as the specific structure and behavior of the target website. As a result, it’s common for a single project to combine automated agents, API requests, and even manual navigation, all under the umbrella of "web scraping."

Taking into consideration the most common web scraping techniques, the concrete tools used in these techniques can be named, which means that they correspond to the most common scenarios.

2.3.1 Static HTML Parsing

This method is used to extract data from web pages where all content is loaded directly within the HTML code, and we need only an HTTP client for fetching the webpage and HTML to extract the required data fields. One of the easiest ways to detect such a webpage is to turn off JavaScript in the browser. If data is still present, then this page can be named as a static (*Scrapfly Web Scraping API | Academy - Scraping Static HTML*, n.d.).

The BeautifulSoup library in Python is widely used for parsing static HTML and XML, as it can navigate the Document Object Model (DOM) tree, which is a structured representation of HTML and XML documents. BeautifulSoup allows users to extract the necessary content from a webpage, remove unwanted HTML markup, and save the cleaned information in a structured format. To retrieve the HTML content initially, the requests library is often used in conjunction with BeautifulSoup. The requests library handles the task of fetching HTML from a webpage by making HTTP requests, allowing users to access web content efficiently.

By using requests, users can retrieve the raw HTML and pass it to BeautifulSoup for parsing and content extraction.

With requests managing the retrieval and BeautifulSoup handling the parsing and cleaning, this combination provides a powerful setup for web scraping. Together, they allow users to navigate through HTML content, extract specific data, and remove any irrelevant tags or structures, creating a clean, structured dataset ready for analysis or storage (*BeautifulSoup Web Scraping*, n.d.; Wieringa, 2012).

2.3.2 Dynamic Content Scraping with headless browsers

Over time, the model of having content on the website in a simple HTML page was moved beyond and was changed to the model of Web applications. The web application is the software that runs on a Web server and allows the management of a website with more advanced features. Basically, the data for the website is completely separate from the actual presentation of the site (Malik, n.d.).

In practice, it means that certain content appears on such types of websites after special interactions, like scrolling, clicking buttons, or waiting for elements to load. Traditional HTML parsing does not capture this dynamically generated content, so headless browsers are used to simulate user interactions and load the necessary data.

For dynamic web scraping, two essential components are required:

1. A headless browser

   Web browser, which operates without a graphical user interface(GUI) and allows automated interaction with websites like any standard browser, but communication does not go through a GUI, but through a command-line interface. Since headless browsers don't have a GUI and don't have to draw visual content, they perform significantly faster. If the website we need to scrap has dynamic content due to a headless browser, we will be able to fully load the page, including HTML, CSS, JavaScript, and handle elements that only appear after the JavaScript code has been executed (*What Is a Headless Browser and Where Is It Used?*, n.d.).

2. A browser automation tool.

   Among the main functions needed for web scraping are interacting with website elements(clicking buttons, scrolling, filling out forms, which is often necessary to reveal dynamic content), waiting for data to load, and navigating the target page to extract necessary data from the DOM. Selenium or Puppeteer are one of the most famous tools, and the principle of their work is simulating real user behavior, interacting with the website, and subsequently finding and extracting elements.

2.3.3 Crawling tools for large-scale data collection

With the increasing need to collect extensive data from multiple pages or sections of a website, crawling tools have become essential for large-scale web scraping. Tools like Scrapy are specifically designed for efficient, high-volume data collection across a wide range of URLs. Scrapy, one of the most popular frameworks for web scraping, enables users to leverage asynchronous crawling, allowing numerous pages to be accessed simultaneously. This approach minimizes delays by avoiding the sequential loading of each page, significantly reducing the time required to gather large datasets.

Scrapy's asynchronous processing is particularly valuable for large-scale projects, as it maximizes resource efficiency and supports high-speed data retrieval. Beyond mere crawling, Scrapy also incorporates a robust data processing pipeline that ensures raw data is cleaned, validated, and structured for easy analysis. Data can be formatted and exported in various ways, including CSV, JSON, or even directly into databases, making it versatile for diverse data storage needs.

Additionally, Scrapy offers flexible selection options through CSS and XPath, which allow for precise targeting of HTML elements, ensuring that only relevant information is extracted. With its powerful combination of asynchronous crawling, error handling, and data processing capabilities, Scrapy stands out as an essential tool for any large-scale web scraping project, facilitating comprehensive data collection and enabling efficient analysis on a broad scale (*Scrapy at a Glance — Scrapy 2.11.2 Documentation*, n.d.).

2.4 Ethical aspect

The practice of web scraping presents numerous ethical challenges that data collectors must carefully consider. One key ethical issue is adhering to the website's terms of service and respecting the robots.txt file, which often indicates areas that the website owner prefers to keep free from automated access. Also, by interacting with websites, it is important to respect their Terms of Service(TOS), which contains more than only the rules for automated access and web scrapers (Mitchell, 2018).

Another ethical consideration is respect for privacy, particularly as it relates to the data being collected. With increased regulatory oversight, such as the General Data Protection Regulation (GDPR) in the European Union and the California Consumer Privacy Act (CCPA) in the United States, web scraping must prioritize user privacy and confidentiality. Scraping personally identifiable information (PII) or any data labeled as private without user consent is not only unethical but may also be illegal. Ethical scrapers should aim to collect only publicly available data, avoid scraping sensitive information, and ensure that the data is handled responsibly, considering both privacy and security concerns.

2.4.1 Fair Use

Fair Use is a type of affirmative defense in the United States law that allows the use of copyrighted material without the necessity to get permission from the copyright holder and is considered relatively simple to understand, since it uses only 4 criteria to understand where a particular use qualifies as fair use:

- Purpose and character of the use

  Non-commercial, educational, or research-related uses are more likely to qualify as fair use, especially when the new use adds value or transforms the original material in some way (e.g., through analysis or commentary).

- Nature of the copyrighted work

  Factual and data-driven materials, such as real estate listings or public records, are more likely to fall under fair use than highly creative or artistic works, which are more strongly protected by copyright.

- Amount and substantiality of the portion used

  Using a smaller portion of a work, or only what is necessary for the specific research goal, strengthens the argument for fair use. Using an entire work is less likely to be considered fair unless justified by necessity.

- Effect on the market value of the original

  If the use does not harm the potential market or reduce the value of the original work, for example, by replacing it or reducing demand, this supports a fair use justification (Crews, 2006; *U.S. Copyright Office Fair Use Index*, n.d.).

3 Practical part

3.1 Business understanding

The United States of America is the third-largest country in the world by total area having the third-largest population, exceeding 334 million people with the first economy in the world by nominal GDP accounting for over 25% of the global economy or 15% if we count by GDP based on PPP ("United States," 2024).

According to the portal Statista.com, the U.S. real estate market is named the second largest in the world with a total value of $113.6 trillion, being only behind Chinese one by $17.6 trillion, but almost tripling real estate market of India, which is on the 3rd places with $39.16 trillion and more than tripling the real estate markets of such developed countries as Germany and Japan, being on the 4th and the 5th places respectively on 2023 (*Real Estate - United States | Statista Market Forecast*, n.d.)

The total value of the market steadily grows year-by-year from 2017, and by 2028, it is expected that the value would reach the number of $142.88 trillion, where $114.7 trillion would be residential real estate and $28.18 trillion would be commercial.

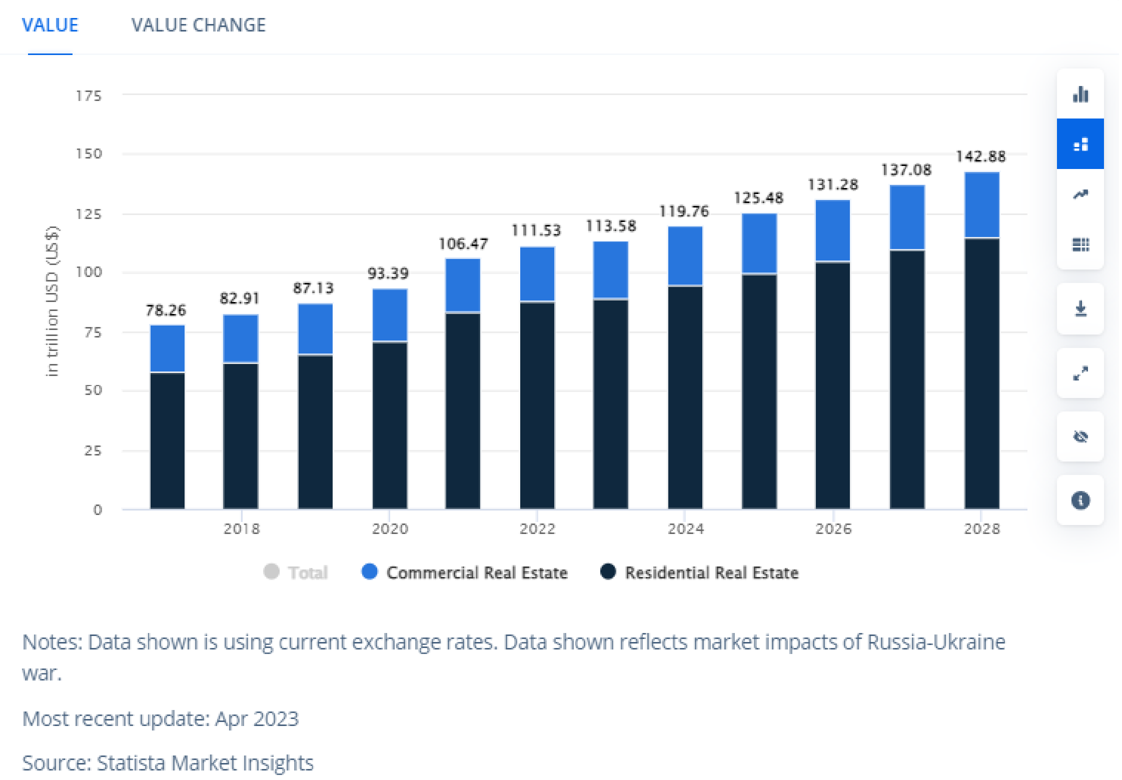

Figure 3 Value of the U.S. real estate market in trillion U.S. dollars (statista.com)

3.1.1 Websites with real estate in the United States

As of 2024, the list of the top 10 most visited real estate websites looked like this:

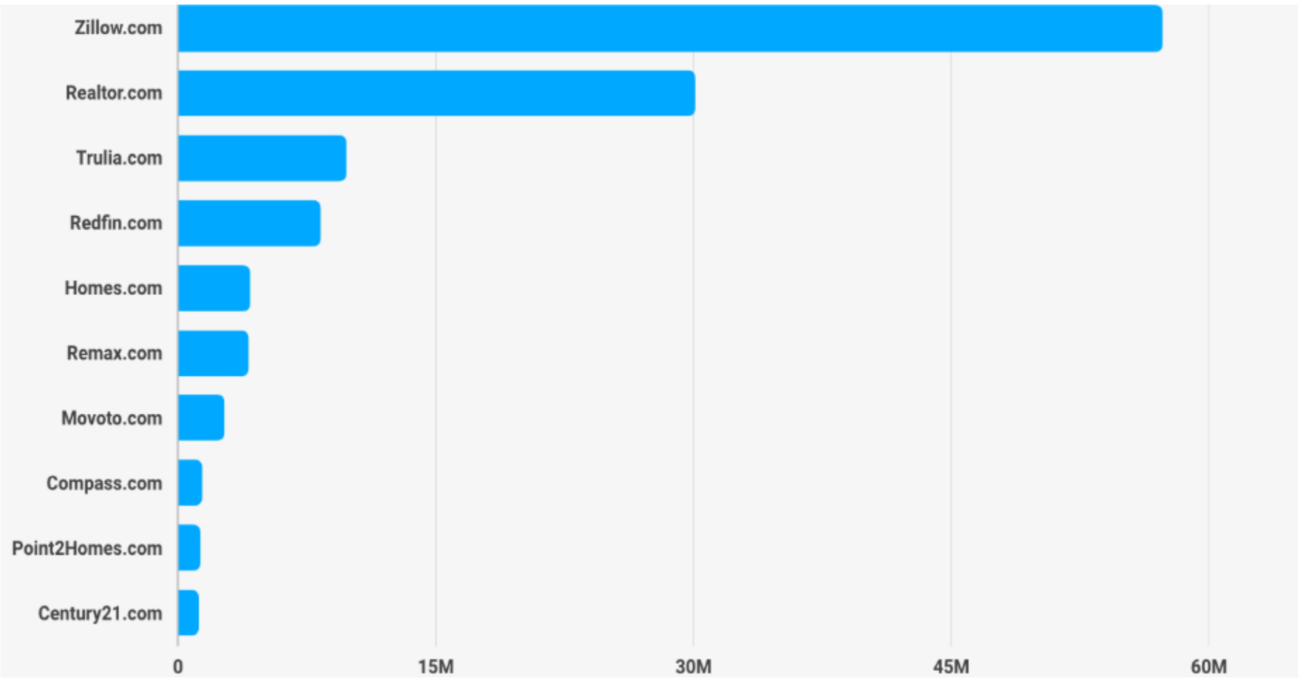

Figure 4 Top 10 most visited real estate websites (www.rubyhome.com)

This ranking counts all data, both mobile, desktop, and web, and it is clearly visible that the site Zillow.com, with its 57 million visits, occupies a large market share. It is interesting to see that the 3rd most visited website from this list, Trulia.com, is owned by the mother company of Zillow as well (Mariotti, 2024).

All websites have similar functionality and allow their visitors to look for real estate advertisements by address, ZIP code, neighborhood, or city, both for sale and for rent. Also, they provide services for successfully selling or renting out the house. For example, on Zillow.com, you can find an agent who would help you with this, or you can read guides and instructions and sell your home by yourself.

For this research, a real estate listing platform was selected based on its accessibility for data collection and its national presence in the U.S. market. While it is not the most visited site in the sector, it ranked within the top ten in recent industry statistics, with over 4 million monthly visits. This platform was chosen due to its relatively open technical structure, which allowed for efficient automated data collection using standard HTTP request libraries. Other platforms, such as the most popular ones, typically employ stricter anti-scraping protections like CAPTCHA systems, which made them unsuitable for this academic study.

3.1.2 Diversity of the United States

The United States has thousands of cities and towns, each with unique real estate dynamics influenced by local geography, culture, and economy. Collecting data from all cities would be an immense undertaking, requiring substantial computational and logistical resources. Given the constraints of this study, such as time and data processing capacity, it was deemed more practical to focus on a carefully selected subset of cities that represent the diversity of the U.S. real estate market.

This focus on diversity acknowledges that regions of the United States can differ significantly in many ways. Factors such as climate, geography, cost of living, population density, and lifestyle all play a role in shaping local real estate markets. The following section highlights these regional differences to provide context for why specific cities were selected for this analysis.

Geographic Verification

The United States has a large geographic variety and includes most climate types. While some areas can have a tropical climate, like Hawaii or South Florida, which is characterized by high temperatures, humidity, and a large number of sunny days during the year, others, like Alaska, are characterized by a subarctic or even polar climate. Besides named types, the large part of the American territory has such types of climates as: humid subtropical, humid continental, desert, and oceanic ("United States," 2024).

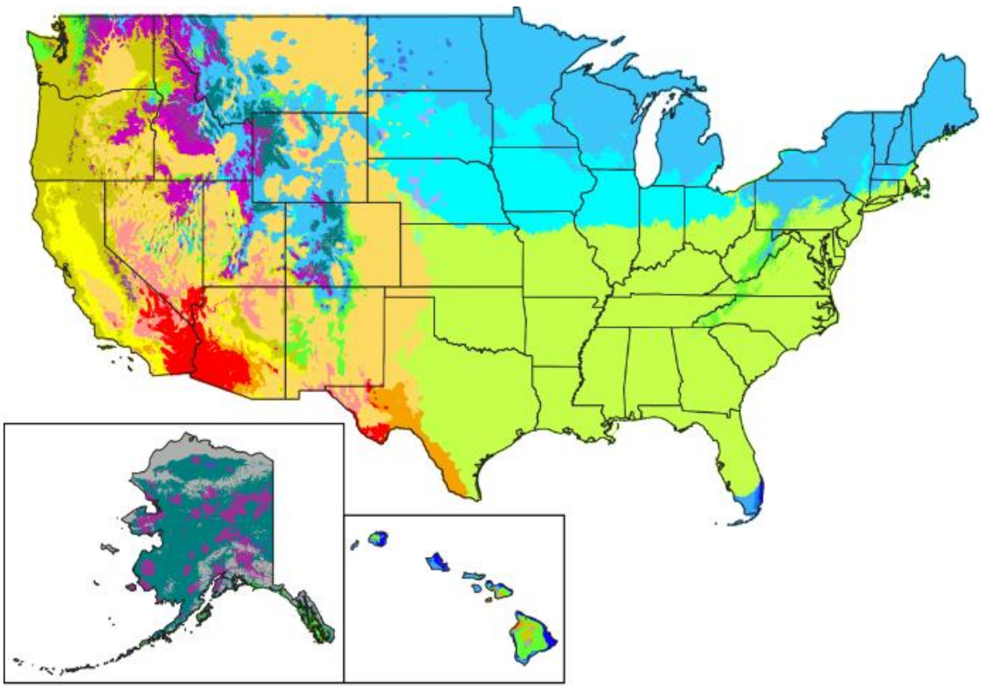

Figure 5 Climate Types in the United States (Climate normals from PRISM Climate group, Oregon State University)

Population and ethnical diversity

The United States has a very diverse racial composition. Cultural influences, rooted in ethnic heritage, traditions, and historical events, have left enduring imprints on local customs, languages, and identities, contributing to the rich tapestry of American diversity.

The table below provides information about the percentage of certain ethnicities in the population of the U.S. as of 2020.

Race	**Percentage**
White	61.6%
Black	12.4%
Asian	6.0%
Native American	1.1%
Pacific Islander	0.2%
Two or more races	10.2%
Other	8.4%

Table 1 Racial composition of the US (US Census Bureau)

Historical patterns and trends

Different regions of the United States have unique historical contexts, including settlement patterns, economic development, and cultural influences.

For example, coastal areas often attracted early settlers due to their access to ports and trade routes, leading to the development of dense urban centres and maritime industries. In contrast, regions with fertile land and favourable agricultural conditions saw the establishment of farming communities and rural settlements. These settlement patterns not only determined the spatial organization of cities and towns but also influenced the development of distinct architectural styles, community traditions, and social identities ("History of the United States," 2024).

These trends are characteristic not only of the period of the beginning or territorial expansion of the United States, which took place in the 18th and 19th centuries, but also to this day. The map below, provided by the US Bureau of the Census, illustrates the change in the population in different states from 1960 to 2000. It is easy to see how uneven the changes were in different states. While the population of some states like North Dakota or Iowa increased insignificantly by 1,6% and 6,1% respectively, or even decreased by 2,8%, as is the case in the state of West Virginia, the other states showed absolutely large population growth. The leaders were Nevada, Arizona, and Florida with their 601.1%, 294,1%, and 222,7% respectively.

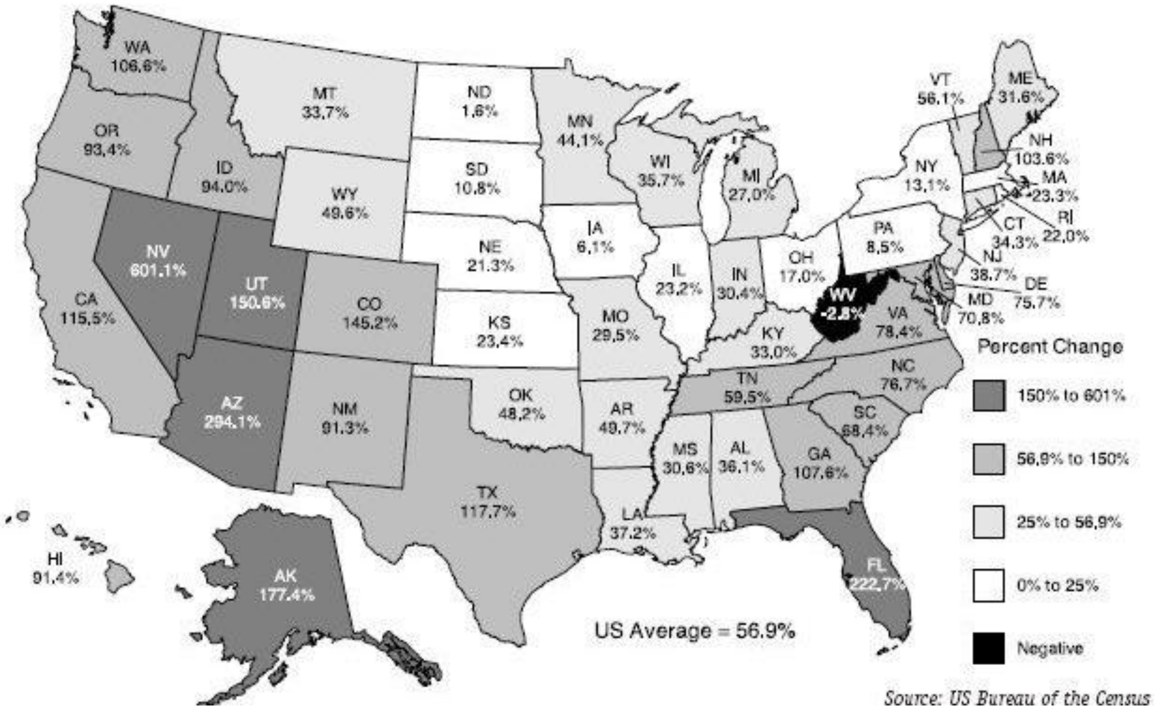

Figure 6 Population change 1960 to 2000 by state (US Bureau of the Census)

3.1.3 Selecting cities for data extraction

Given the scale and diversity of the United States, analyzing the real estate market across all cities would be an immense challenge, requiring vast computational and logistical resources. To create a manageable and representative dataset, I decided to choose at least 20 cities with a population of over 500,000, which would belong to different regions of the United States. For defining regions of the US, I will focus on the maps made by different organizations.

One of the most famous and commonly used region definitions for data collection and analysis since 1950 is statistical regions defined by the United States Census Bureau (Bhattacharyya & Timilsina, 2009).

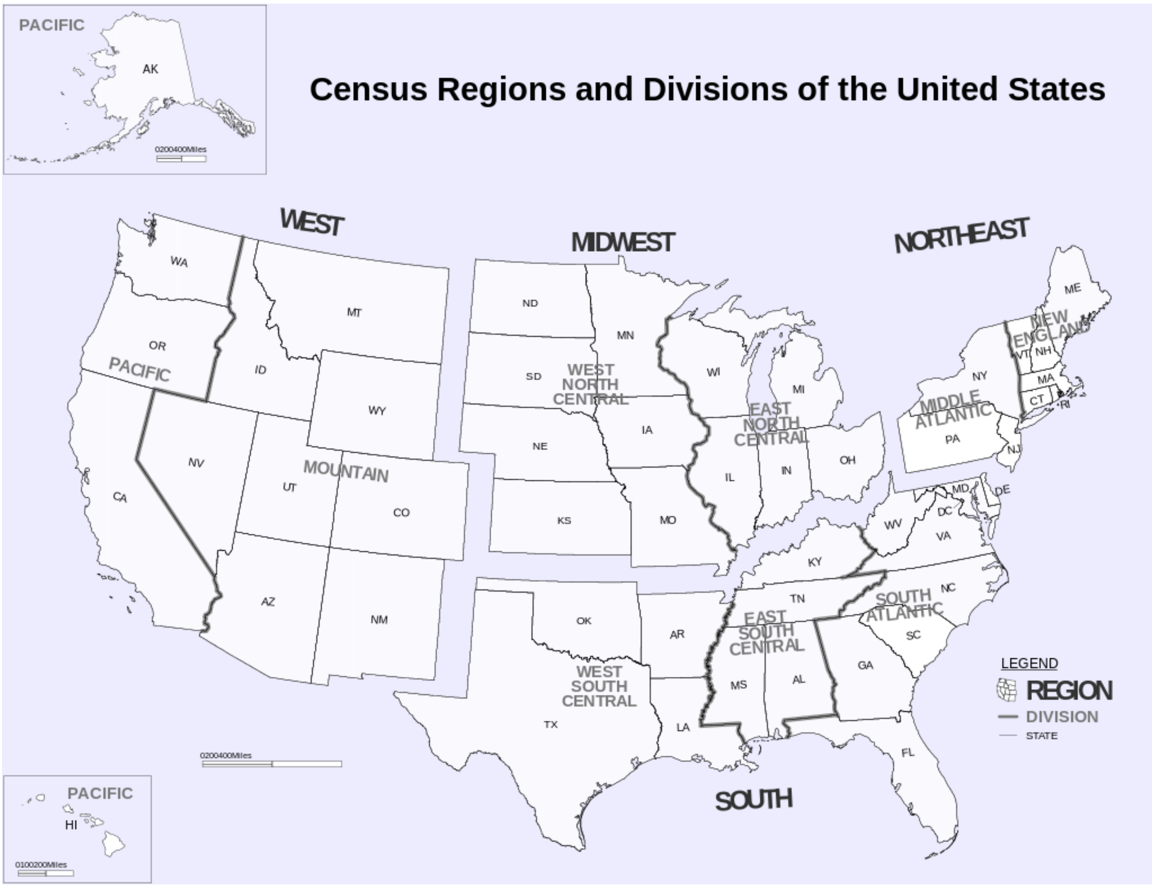

Figure 7 Census Regions and Divisions of the United States(U.S. Census Bureau)

This map defines 4 regions and 9 divisions within the regions.

One of the main reasons why this model is being criticized is that it defies borders for the regions it uses administrative borders and fails to account for cross-border regions that extend beyond state boundaries. For example, it groups the entire states of California, Oregon, Washington, Hawaii, and Alaska into a single region, disregarding their significant differences in climate, geography, economy, and cultural identity. California alone spans multiple climate zones, ranging from arid deserts in the southeast to the Mediterranean climate of the coast and the snowy mountain regions of the Sierra Nevada. Similarly, Alaska's subarctic and polar climates differ dramatically from Hawaii's tropical environment.

Another example is that huge metropolitan area on the East Coast, what is called Boswash and includes such big cities as: Boston, New York, Philadephia, Baltimore and Washington D.C., being hub of urbanization, economic growth, and social and political importance, are separated and belong to different divisions, which are not that economically strong and urbanized (Wright et al., 2022).

One more popular system of dividing the country into twelve districts with a central Federal Reserve Bank in each district, for comparison of economic data, or according to the Office of Management and Budget, which published in 1969 a list of ten "Standard Federal Regions". Several agencies continue to follow that system, including, for example, the

Environmental Protection Agency and the Department of Housing and Urban Development ("United States Department of Housing and Urban Development," 2024; US EPA, 2013).

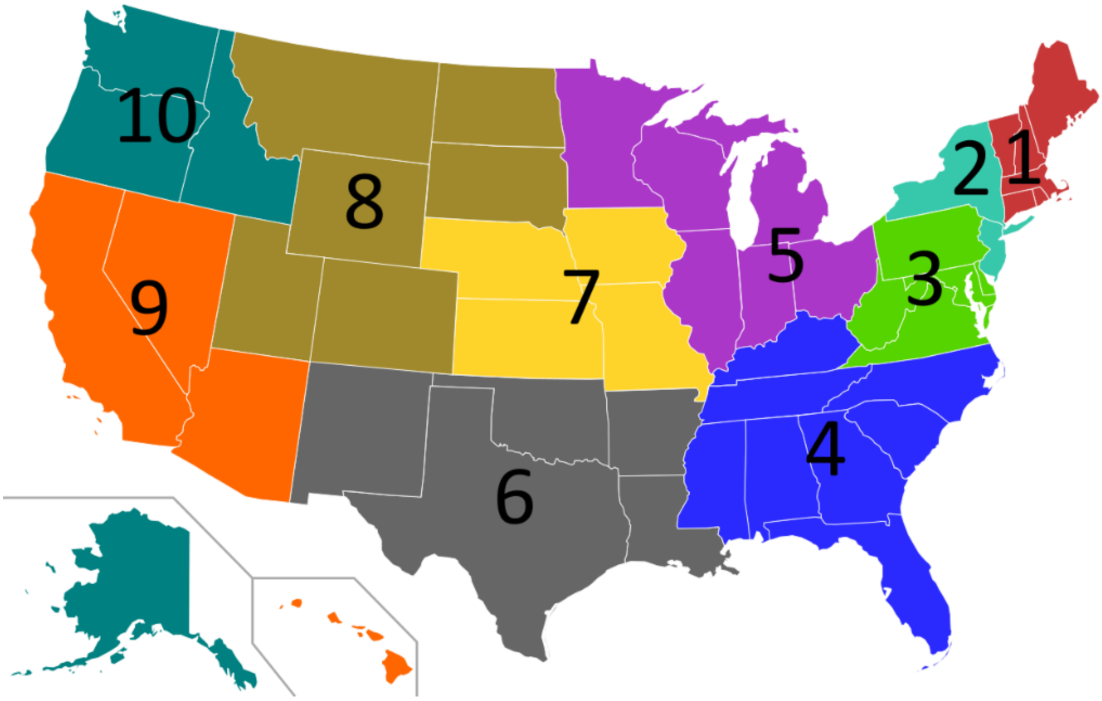

Figure 8 Regions of the United States(US EPA)

Nevertheless, due to the large diversity in different factors, there is no perfect system for uniting different parts of the U.S. into regions, considering all of them. While official classifications provide a foundation for regional analysis, they often face criticism for their rigidity and lack of adaptability. This has motivated individuals to create alternative maps, like those made by the Reddit community, which aim to reflect cultural, economic, or geographical nuances often missed in formal frameworks. Such maps tend to be more detailed, as they are created with input from communities of individuals residing in various parts of the country. This process often incorporates localized knowledge and perspectives, resulting in representations that reflect regional characteristics in greater depth compared to standardized official classifications. You can see an example of such a map below.

Figure 9 Cultural Regions of the US (Source: Reddit)

This map contains 6 main Regions, what is:

- Pacific
- Frontier
- Midwest
- Northeast
- The South
- Caribbean U.S

Inside the main regions, there are small subregions, the total number of which is 46.

Based on the community map and taking into consideration studies presented earlier, I selected 23 urbanized areas from different regions of the United States for scraping real estate listings and conducting a subsequent market analysis.

The next table provides information about the regions and cities that were chosen for the future analysis. The attributes “Region” and “Subregion” are filled based on the zoning of Reddit community map:

City	Subregion	Region
Seattle	Cascadia	Pacific
San Francisco	SF Bay Area	Pacific
Sacramento	Central Valley	Pacific
Los Angeles	Southern California	Pacific
Anchorage	Alaskan Maritime	Pacific
Honolulu	Hawaii	Pacific
Las Vegas	Classic Southwest	Frontier
Salt Lake City	Mormon Corridor	Frontier
Oklahoma City	Lower Great Plains	Frontier
Houston	Texas Heartland	Frontier
Dallas	Texas Heartland	South
New Orleans	Gulf Coast	South
Atlanta	Deep South	South
Orlando	Central Florida	South
Charleston	Southern Tidewaters	South
Nashville	Midsouth	South
Miami	South Florida	Caribbean U.S.
Minneapolis	Upper Midwest	Midwest
Chicago	Chicagoland	Midwest
Columbus	Ohio River Valley	Midwest
Pittsburgh	Northern Appalachia	Northeast
New York	NYC Metro	Northeast
Boston	Maritime New England	Northeast

Table 2 Selected cities for the analysis (author)

3.2 Data collection

For scraping, a U.S.-based real estate platform ranked among the top ten most visited websites in the sector was selected. The platform was chosen due to the absence of technical barriers such as CAPTCHA or anti-bot systems, which made it suitable for automated data collection. All collected data was used exclusively for academic and research purposes. The author has no commercial intentions, and the data was processed strictly in accordance with the fair use principle. Scrapped data won’t be used for any commercial goal by the author and was chosen exclusively for research goals.

3.2.1 Web scraping

Example of the code, which was used for scraping data from real estate listings for the city of Los Angeles, with an explanation of how it was created step-by-step, which will be provided below.

For scraping a website, I requests and Beautiful Soup libraries will be used.

First of all, we define a function "get_data", which takes the "base_url" argument. Inside the function, the headers dictionary is defined, which contains headers to simulate a browser when making requests. This dictionary specifies User-Agent, which is a string used to identify the type and version of the browser used to make the request. In our case, the User-Agent for Chrome on Windows 10 is specified.

```
import requests
from bs4 import BeautifulSoup


def get_data(base_url):
    headers = {
        "user-agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/121.0.0.0 Safari/537.36"
    }
```

Code 1 Initial setup of the web scraping function (author)

It is important to define the difference between the parameters "url" and "base_url" in the code. "Base_url" is the link on the webpage with list of listings for certain city or county, while "url" is used in the cycle "While true" and will be changed by turning pages, since on the website on one web page only 24 real estate listings, while we will scrape at least 1000 listings for each city.

By creating the "listing_name" variable, each single parsed listing will be stored by its code (for Los-Angeles, the code "LAC" was chosen) and id "listing_number", which after each iteration will be increased by 1.

Using context manager "with open", we firstly create and save to a separate folder for each city the HTML code of each listing to use this code for sending requests, and then read it into the variable "src". After this, we are looking for the first element "div" with a specific class that holds all relevant listing data. "Listing_data" will then be used for looking for separated details of the listing by looking for them by HTML code.

Since most of the required information for data analysis is located inside the listing, which can be opened by clicking on it, we have to extract data not only about price, number of beds, baths and square feet, but also data, located inside listing itself, for example, does the real estate object have HOA fee, Cooling or Heating systems and so on.

```
while True:
```

```
    url = f"{base_url}/page-{page}"
    req = requests.get(url, headers=headers)
    listing_name = f"SETL{listing_number}"
    listing_number += 1


    with open(f"data_LAC/{listing_name}.html", "w") as file:
        file.write(req.text)

    with open(f"data_LAC/{listing_name}.html") as file:
        src = file.read()

    soup = BeautifulSoup(src, "lxml")
    listing_data = soup.find("div", class_="…")


...
base_url = "https://www.remax.com/homes-for-sale/ca/los-angeles-county/county/06037"
get_data(base_url)
```
Code 2 Loop for iterating through listing pages and saving individual HTML files for each property (author)

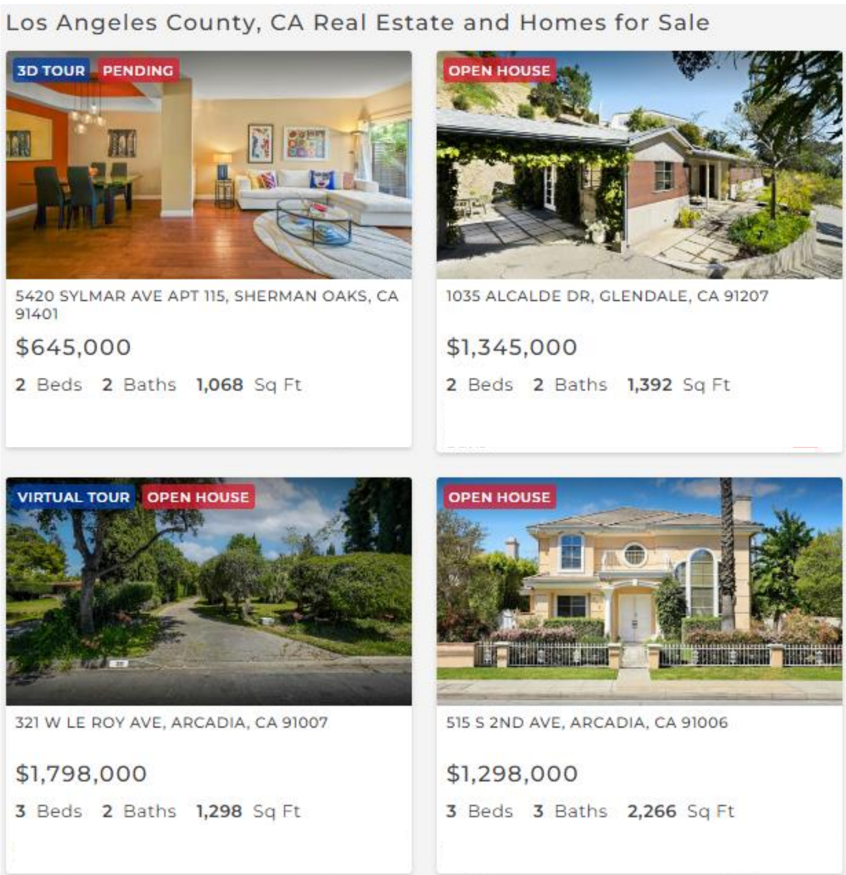

Figure 10 Layout of webpage with listings (author)

Using a class that contains a link to single listings, we are looking for them on each page by changing url in the cycle

```
While True:
...
soup = BeautifulSoup(src, "lxml")
listings = soup.find_all("div", class_="card-full-address cursor-pointer")
print(listings)
```

Code 3 Parsing the HTML page and extracting all individual real estate listing blocks using their class name (author)

Due to the fact that the name of the div class and the name of the h4 class for different values showed on the website are completely same, in order to avoid a situation where, of the searched parameters, the code will give as an output only the first values showed on the page it can find (in our case, the number of bedrooms), we use a CSS-selector, create list “values” and put data in text format into it, after which we assign got data to our parameters like “bedrooms”, “baths”, “acres”, etc. Since the site does not force users publishing listings to provide all information, advertisements may be incomplete, and some information may be missing. A “try-except” loop will be used to solve this problem. If the data is missing, the “null” value will be assigned to our parameter.

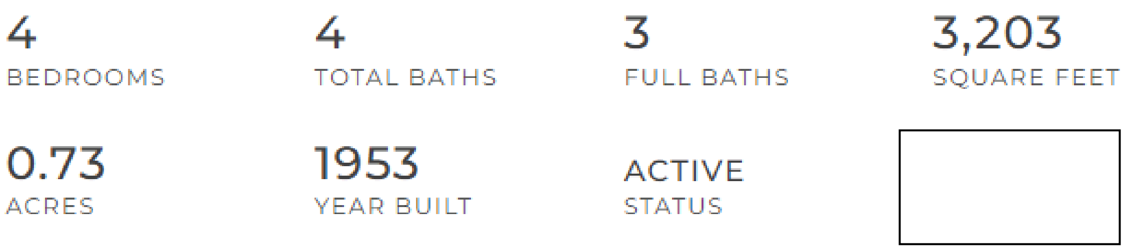

Figure 11 Layout of needed attributes on the webpage (author)

```
▼<div class="listing-detail-stat-container w-1/2 md:w-1/4" data-v-
  3916bc1a> flex
    <h4 class="listing-detail-stat h2" data-v-3916bc1a> 4 </h4>
    <h6 class="listing-detail-stat-label" data-v-3916bc1a> Bedrooms
    </h6>
  </div>
▼<div class="listing-detail-stat-container w-1/2 md:w-1/4" data-v-
  3916bc1a> flex  == $0
    <h4 class="listing-detail-stat h2" data-v-3916bc1a> 4 </h4>
    <h6 class="listing-detail-stat-label" data-v-3916bc1a> Total Baths
    </h6>
```

Code 4 Repeated div containers with identical class names used to display individual listing parameters (website)

```
h4_elements = soup.select('div[class="listing-detail-stat-container w-1/2 md:w-1/4"]
h4[class="listing-detail-stat h2"]')
```

```
values = []

for h4 in h4_elements:
    values.append(h4.text)

bedrooms, baths,full_baths, square_feet, acres, year_built = values
```

Code 5 Extracting individual property attribute from structured HTML blocks using CSS selectors (author)

In some cases, objects in HTML code cannot be found using the "find" method from the BeautifulSoup library, which identifies them as a variable with the value "None". In this case, we look for the class above and write out a whole piece of code, after which we remove unneeded lines in the code. This method was used for getting the attribute "price"

```
price = listing_data.find('h4', class_='md:text-right h3').get_text(strip=True)
```

Code 6 Extracting the property price from a listing page using a specific HTML tag and class (author)

```
Price: <h4 class="md:text-right h3" data-v-9a7568f8="">
            $1,679,000<!-- --></h4>
```

Figure 12 HTML structure showing the price of the property inside an <h4> element with class md:text-right h3.

After the code is written, we can only parse ads and detailed data from ads only from the first page of the site. So that we can parse advertisements from the following pages, we indicate the initial possible page at the beginning of the code and create an endless loop while true, in which we will sequentially go through the pages of the site specified in the base_url.

The next code is used to extract information about the garage property type, the garage spaces, and the total number of parking slots.

```
data_wrappers = soup.find_all('div', class_='data-wrapper')

data_values = {}

for data_wrapper in data_wrappers:
    data_blocks = data_wrapper.find_all('div', class_='w-full sm:w-1/2 mb-6')

    for block in data_blocks:
        try:
            data_name = block.find('div', class_='data-name').text.strip()
            data_value = block.find('div', class_='data-value').text.strip()
            data_values[data_name.lower()] = data_value
        except AttributeError as e:
```

```
            print("Error", e)
            continue

property_type = data_values.get("property type")
garage_spaces = data_values.get("garage spaces")
parking_total = data_values.get("parking total")

property_type = "null" if property_type is None else property_type
garage_spaces = "null" if garage_spaces is None else garage_spaces
parking_total = "null" if parking_total is None else parking_total
```

Code 7 Extracting property type, number of garage spaces, and total parking slots from structured HTML blocks (author)

The peculiarity is that the values that we need to get are in the div class data-value in the page code, which will appear if there is a div class data-name with a specific value. Then div classes are being wrapped in other div classes, building a hierarchy.

To find the needed information, we need to use the find_all method from BeautifulSoup, and then we create a dictionary to store data values with keys corresponding to the data name

Then we find elements with the class "w-full sm:w-1/2 mb-6" and extract values from each block in a loop.

By default, the get() method, if the key was not found, returns the value 'None', but in our case this can be misleading, since on the site itself in the interior features section, for example, the user can enter any value, and by entering 'None' there we will not know what was entered whether it is the user or the data is simply missing. In the absence of dates, we cannot directly say that, in this case, there are simply no interior features. Perhaps they were forgotten and indicated. So we change None to null

During research, I have figured out that some information I was planning to scrap is not in the saved HTML code of the listing. Analysing the website network, I have figured out that to get this information, the site uses the GET method on the api url.

e

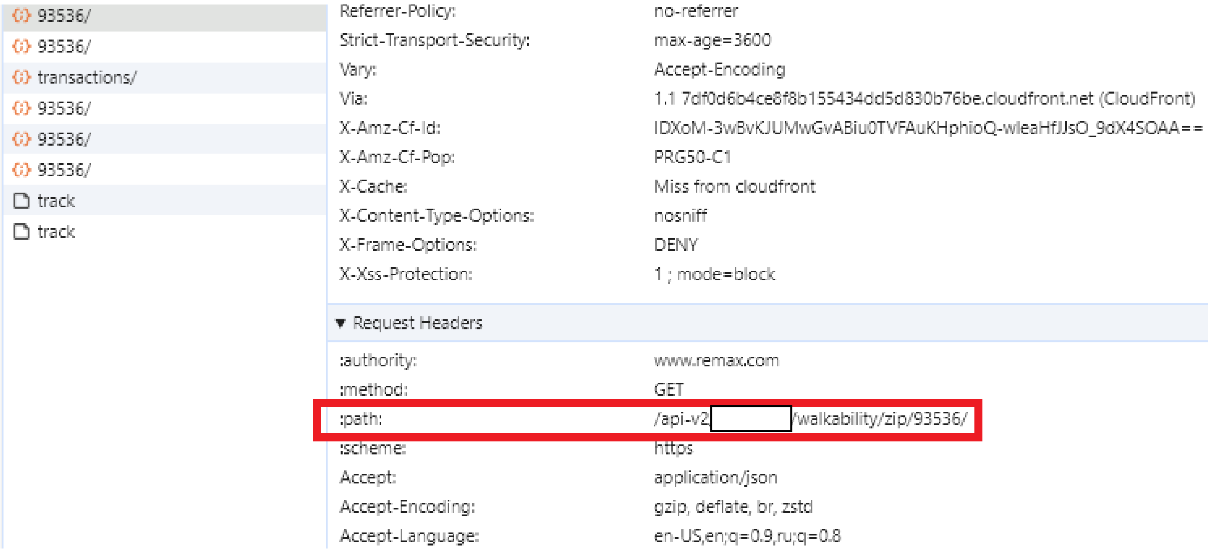

Figure 13 Network tab in Google Chrome DevTools: API Request for Walkability data (website data)

Figure 14 API Request for Demographics data (website data)

It is interesting to notice, that by API request we can get a lot of data, but the site shows only a few parameters, for example: the convenience of the area for pedestrians and drivers and demographics: the number of people who own housing in the area, the number of people renting housing in the area and the number of households with children. However, through the API, we can get more information that the site does not display, for example, the number of people of different races.

We can find the zip code of the site at the top of the page and insert our code into a loop that processes each real estate advertisement separately. With zip-code, we will pass the zip code to the zip-code variable, which we will use to access the api.

Next, using the api requests that the website sends, we write code that will access it and, using the GET request, receive data in case of a successful response from the API, which we can understand by the status code 200. If the response is successful, the code converts the JSON response into a dictionary and extracts the value, in this case, overall_rating, from the dictionary

```
api_walk_url = f"https://www....com/api-v2/.../walkability/zip/{zip_code}/?"
api_walk_response = requests.get(api_walk_url, headers=headers)
```

```
if api_walk_response.status_code == 200:
    walk_data_api = api_walk_response.json()
    walk_overall_rating = walk_data_api.get('overall_rating', '')
    print("Overall rating:", walk_overall_rating)
```
Code 8 Retrieving walkability data by ZIP code via an API request (author)

To have data in readable format it was decided to put all scrapped data to CSV file. Next lines of code are responsible for this.

```
def get_data(base_url):
...
with open('SETL_data.csv', 'w', newline='') as csvfile:
    fieldnames = ['Listing_ID', 'City', 'County/Parish']
while True:
for listing_url in enumerate(listings_urls):
with open('SETL_data.csv', 'a', newline='') as csvfile:
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
    writer.writerow(
    {'Listing_ID': listing_id, 'City': city, 'County/Parish':county_or_parish}
```
Code 9 Saving collected real estate data to a CSV file (author)

3.3 Data understanding

In the Data Understanding phase, the focus is on analyzing individual attributes by examining their distribution, data types, and value ranges to understand their structure and relevance. This step also involves identifying missing values, anomalies, and outliers that could potentially impact the analysis or modeling process. By addressing these issues early, we ensure the dataset is clean and suitable for further analysis.

As a result of using the script in the previous step, we have received CSV files with data bout listings in the chosen cities. The total number of CSV files is the same as the total number of cities we took for analysis – 23.

By using the next command, I have combined all of them into one CSV file by the name “all-csv-files.csv” with a total amount of listings we got for analysis of 84984. Before using the command, each .csv file except the first one, which was the file with data for the city of Anchorage, was opened, and the headline was removed. All attributes in the headline were broken to standard: upper letter for attributes, which name contains no more than 2 words, in the attribute “Parking total” name was changed to “Parking Spaces” by analogy with “Garage Spaces”.

```
copy *.csv all-csv-files.csv
```
Code 10 Merging csvs (author)

In some cases, I collected data at the County level rather than strictly within city limits. This approach was necessary for cities with either small administrative boundaries or unclear delineation between urban and suburban areas. Using County-level data ensured more comprehensive coverage of the actual urbanized region and helped avoid sample bias due to overly narrow geographic constraints. The table below provides information about the population of the scraped area and the number of listings saved to the dataset.

City	**County**	**Population**	**Number of received listings**
Anchorage	Anchorage Municipality	291 247	1674
Atlanta	Fulton county	1 066 710	3656
Boston	Suffolk county	797 936	1429
Columbus	-	905 748	3647
Denver	Denver county	715 522	3559
Honolulu	Honolulu County	1 016 508	3716
Houston	-	2 304 580	4679
Charleston	Charleston County	408 235	3771
Chicago	-	2 746 388	3582
Los Angeles	Los Angeles County	10 014 009	3450
Las-Vegas	-	641 903	1971
Miami	Miami Dade County	2 701 767	5530
Minneapolis	Hennepin county	1 281 565	3751
Nashville	Davidson county	715 884	4359
New Orleans	-	383 997	4094
New York City	-	8 804 190	8629
Oklahoma city	-	681 054	1431
Orlando	-	307 573	4116
Pittsburgh	Allegheny county	1 250 578	5519
Sacramento	Sacramento county	1 585 055	3670
Salt Lake City	Salt Lake City	199 723	3667
Seattle	-	737 015	3667
San Francisco	San-Francisco County	873 965	1417
Total			84984

Table 3 Received listing by each city(Bureau, n.d.)

3.3.1 Description of attributes

The total number of attributes is 64. The table below provides information about all of them, including the name of the attribute, the data type of the attribute, the number of missing values, and the explanation.

Attribute	**Data type(Pandas)**	**Definition**
Listing ID	Int64	ID of the scrapped listing in order

Location Request	Object	The name of the city was used as input on Remax.com to scrape listings. The attribute was made by the author
City	Object	Name of the city according to the listing owner
County/Parish	Object	Name of the county or parish according to the listing owner
Subdivision	Object	Name of the subdivision according to the listing owner
Bedrooms, Baths, Full Baths	Object	Number of bedrooms, baths, and full baths respectively
Square Feet	Object	Total interior living space of a property in square feet
Acres	Object	The total size of the land belonging to the property in acres
Year Built	Object	The year when the real estate object was built
Price	Object	Price of the real estate
Price/Sq ft	Object	Price for each square foot
HOA Amount	Object	Amount of HOA payment
HOA Features	Object	Amenities and services provided by the homeowners' association
HOA Frequency	Object	How often are HOA fees paid
HOA Includes	Object	What the HOA fees cover, such as maintenance, landscaping
Agency Fee	Float64	
Cooling Y/N Heating Y/N	Object	If there is cooling or heating
Property Type	Object	Type of the property
Property Sub Type	Object	Subtype of the property
Garage Spaces Parking Spaces	Float64	Spaces in the garage or on the parking lot
Cooling type Heating type	Object	Type of cooling or heating
Interior Features Exterior Features	Object	What features interior or exterior have
Zip Code	Int64	Zip-code of the district where the listing is located
Walk Rating Drive Rating	Float64	Walking and driving ratings, where 0.0 is – lowest possible, 5.0 is – highest possible
Owners % Renters %	Float64	Percentage of owners and renters of real estate in the district
People Total	Int64	Total number of people where real estate is located
People Density	Float64	People density in the district where the listing object is located
White collar jobs Blue collar jobs	Float64	Percentage of people living in the district doing white/blue-collar jobs

Age under 9yo % Age 10_19yo % Age 20_34yo % Age 34_44yo % Age 45_64% Age 65_84yo%	Float64	Percentage of people of the selected year ranges in the district where the listing object is located.
Rent under 400 % Rent 400_799 % Rent 800_1249 % Rent 1250_1999 % Rent over 2000 %	Float64	Percentage of the amount of rent, which is paid in the district where the listing object is located
Male % Female %	Float64	Percentage of males and females living in the district where the listing object is located
White pop % Black pop % NatAm pop % Asian pop % Other pop % Multi pop %	Float64	Percentage of each nationality living in the district where the listing object is located
No HS% Yes HS%	Float64	Percentage of people who ended/did not end high school, living in the district where the listing object is located
Bachelors %	Float64	Percentage of people with a college degree living in the district where the listing object is located
Vehicles per unit %	Float64	Percentage of vehicles per unit in the district where the listing object is located
N of families %	Float64	Percentage of people who are part of the family(non-single people)

Table 4 Description of attributes in the dataset (author)

It can be noted, that most data, that should have numeric type such as: Bedrooms, Baths, Full Baths, Square feet, Acres, HOA Amount, Year Built, Price, Price/ Sq Ft were read as string data type by Python. The reason is, that they contain values and signs like “$”, “—”

3.3.2 Duplicate values

Duplicate records represent identical listings that appear more than once in the dataset. Their presence can inflate the dataset size and potentially lead to misleading insights if not addressed. Identifying and understanding these duplicates is an essential step prior to any reliable data analysis.

To detect duplicate values next Python code was used.

```
import pandas as pd
```

```
file_path = r"C:\Users\germa\PycharmProjects\webscrap\excels\all-csvs_excel.xlsx"
df = pd.read_excel(file_path)

# Excluding 'Listing ID' from the comparison
df_no_id = df.drop(columns=["Listing_ID"], errors="ignore")

# Converting all values to strings and normalizing them to lowercase
df_normalized = df_no_id.astype(str).apply(lambda x: x.str.strip().str.lower())

# Replacing 'nan' strings with a consistent placeholder
df_normalized = df_normalized.replace("nan", "missing")

# Combining each row to a single string
row_strings = df_normalized.apply(lambda row: " | ".join(row.values), axis=1)

# Identifying duplicates based on the row strings
duplicate_mask = row_strings.duplicated(keep=False)
```
Code 11 Detecting duplicate listings based on normalized row (author)

The total number of duplicates is 11,773. Such a big value might be caused by the following problems: the website allows the same real estate object to be posted a few times. Moreover, after reaching the last listing page for the selected city, it starts copying ads from the first to the last.

To be able to look closely at the duplicate rows separate dataset with duplicates was created.

```
# Extracting duplicate rows
duplicates_df = df[duplicate_mask]

# Saving the duplicates to a new Excel file
output_path =
r"C:\Users\germa\PycharmProjects\webscrap\excels\step0001_duplicates_dataset.xlsx"
duplicates_df.to_excel(output_path, index=False)
```
Code 12 Extracting and saving duplicate listings to a separate Excel file (author)

Duplicate listings by "Property Type": Farm – 993 of 1,746, Land – 4965 of 8,497, Residential – 4,957 of 67,466, Residential Income – 858 of 7,264.

Although the formal removal of duplicates is typically part of the Data Preparation phase, their presence was noted during Data Understanding as they can considerably distort preliminary statistics, particularly those related to missing values and frequency distributions. For the purpose of clearer exploratory analysis, a temporary filtered version of the dataset, excluding exact duplicates (based on all attributes except Listing ID), was used in parts of this section.

```
# Create a mask for unique rows (keep only the first occurrence)
unique_mask = ~row_strings.duplicated(keep='first')
```

```
# Filter original DataFrame using the mask
df_deduplicated = df[unique_mask]
```
Code 13 Removing exact duplicates by keeping the first occurrence only (author)

From each group of duplicated values, only 1 was left. From 15,785 duplicated lines, 12, 507 were removed, and 72,477 listings will be used for the overview of the missing values. This high number of removed entries indicates that several individual listings were repeated multiple times in the original dataset — a common outcome in datasets obtained through web scraping or repeated data exports.

3.3.3 Missing Values

Since our dataset consists of secondary data, it contains a significant number of missing values. The website from which the data was scraped does not enforce strict requirements for completing all fields when creating a listing. As a result, many attributes contain missing or incomplete values. Because at this point duplicates haven't been removed from the dataset, and this overview in the part "Data Understanding" is based on the initial raw dataset, duplicate records have not yet been removed. Therefore, some missing values may appear inflated due to repeated listings.

During the missing value analysis, it was observed that a significant portion of missing values across several attributes is not random, but rather follows structural or semantic patterns. Such missing values, which depend on other attributes, are referred to as conditional missing values.

"Property type" plays an important role in the analysis. Since the study is aimed at the analysis of residential objects attributes, which match the residential real estate market, the following attributes were chosen(bedrooms, baths, HOA-attributes, etc.). Because of these reasons, listings with "Property type "Land" have a missing value in all attributes except "Price", "Agency Fee", and attributes providing data about the area where the real estate object is located by its zip code. The same works for listings with property type "Farm", where selected attributes have more than 90% missing values.

Attribute	**Missing value**	**Total number**	**Number in %**
Bedrooms	"—"	774	97.48%
Baths	"—"	777	97.86%
Square Feet	"—"	781	98.36%
Year Built	"—"	763	96.10%
Cooling Y/N	No Info	790	99.50%
Heating Y/N	No Info	791	99.62%
HOA Amount	No Info	775	97.6ë%
Interior Features	Null	792	99.75%
Exterior Features	Null, "See Remarks"	794	100.00%

Table 5 Missing values of "Property Type" = "Farm" (author)

Residential income real estate is the real estate that is aimed to be rented out, and it is mostly houses where multiple households can live, motels, or separate buildings. This list of property contains fewer missing values in percent than farms, but relatively more than residential.

Attribute	Missing value	Total number	Number in %
Bedrooms	"—"	2048	31.95%
Baths	"—"	1956	30.51%
Square Feet	"—"	1950	30.42%
Year Built	"—"	286	4.46%
Cooling Y/N	No Info	2725	42.51
Heating Y/N	No Info	1074	16.76%
HOA Amount	No Info	6101	95.18%
Interior Features	Null, "See Remarks", "Other"	4875	76.05%
Exterior Features	Null, "See Remarks", "Other	4387	68.44%

Table 6 Missing values of "Property Type" = "Residential Income" (author)

For comparison, for "Property Type" = "Residential," the number of missing values is presented in the table:

Attribute	Missing value	Total number	Number in %
Bedrooms	"—"	1181	1.91%
Baths	"—"	302	0.49%
Square Feet	"—"	3540	5.73%
Year Built	"—"	1107	1.79%
Cooling Y/N	No Info	10144	16.43%
Heating Y/N	No Info	7277	11.79%
HOA Amount	No Info	34616	56.07%
Interior Features	null, "Other interior features", "Other", "See Prop Desc Remarks", "See Remarks"	24341	39.43%
Exterior Features	Null, "See Remarks", "Other	32351	54.40%

Table 7 Missing values for "Property Type" = "Residential" (author)

Since this study focuses on analyses of Residential Real Estate and we can't compare it with listings, representing land for sale, farms, and residential real estate, we will make detailed analyses of listings with property type "Residential", "Manufactured in Park", "Single Family Residence / Townhouse". The last 2 values are represented only by 4 listings in the whole dataset, and can also be considered as residential real estate, and these values will be changed at the step of data preparation. The total number of listings taken for detailed analysis is 61,743.

The attribute "Property type" plays a key role in the dataset. It explains what type of property was scrapped and contains next values: "Residential", "Land", "Farm", "Manufactured in

Park", "Residential Income", "Single Family Residence / Townhouse". The study aims to investigate residential real estate, and based on this, the information that needed to be extracted was selected. Land, Residential income(on chosen real estate website it means real estate, which is used for commercial reasons: hotels, motels, houses divided into separate apartments for renting them out) and all other types of property except "Residential" and "Single Family Residence / Townhouse" aren't goal of our analysis and such listings will be excluded at the step of Data Preparation. A total number of these listings is 17516, and they won't be considered for analysis of missing values, since they can highly affect the results and cause the removal or dealing with listings, which will be removed at the step of Data Preparation.

None of the chosen cities has lost a critical quantity of listings, and all of them will be taken for further steps of analysis.

The table below provides information on what null values each attribute contains. Attributes, which are not in the table, do not contain any missing values. Also, it is important to note that attributes "Interior Features", "Exterior Features", "HOA Includes", "HOA Features", "Cooling Type", and "Heating Type" are multi-label categorical attributes, which means that each values store several categories, which are separated by a comma. Moreover, such categories might present useless information("Unknown", "Other", "See Remarks", while formally they can not be classified as missing, on the step of data preparation, they have to be changed to "null". In the case of multi-label categorical attributes in this research, a value was considered missing only if none of the individual categories (separated by commas) provided meaningful information. For example, a string such as "Other, See Remarks" was treated as missing, whereas "Other, Walk-In Closet" was retained, since it contains at least one informative element.

Attribute	**Type of missing value**	**Total number**	**Number in percents**
City	null	3	0.00%
County/Parish	null	617	1.00%
Subdivision	null, "0", ".", "¤arch Creeks Estates"	22579	36.57%
Bedrooms	"—"	1181	1.91%
Baths	"—"	302	0.49%
Full Baths	"—"	895	1.45%
Square Feet	"—"	3540	5.73%
Acres	"—"	16925	27.41%
Year Built	"—"	1109	1.80%
Price	null, "0"	18	0.03%
Price/Sq Ft	No Info	3563	5.77%
HOA Amount	No Info	34620	56.07%
Cooling Y/N	No Info	10146	16.43%
Heating Y/N	No Info	7279	11.79%

Garage Spaces	null	29148	47.21%
Parking Spaces	null	18234	29.53%
Agency Fee	null	9013	14.60%
Interior Features	null, “Other”, “Other Interior Features”, “See Remarks”, “See Prop Desc Remarks”	24343	39.43%
Exterior Features	null, “Other”, “See Remarks”	32353	52.40%

Table 8 Missing values in the dataset(Part 1) (author)

While most missing values in the dataset can be detected using null or placeholder values (e.g., “—“ certain attributes require a semantic approach.

For instance, take the attribute “Cooling Type”. If the related attribute “Cooling Y/N” is "Yes" and “Cooling Type” is null, this is a true missing value, since cooling is present, but the type is not specified. On the other hand, if “Cooling Y/N” is "No" and “Cooling Type” is also null, the missing value is semantically justified and should not be counted as missing — cooling is explicitly absent.

Another example is HOA-attributes. If “HOA Amount” > 0, values like “None“, „Not Applicable”, etc. in attributes “HOA Features”, “HOA Includes”, “HOA Frequency” are meaningless and must be changed to null.

Attribute	**Condition**	**Type**	**Inaccurate values**	**Number**	**In %**
HOA Includes	HOA Amount >0	Missing	“No Maintenance Included”, “Other”, null, “See Remarks”	9769	
		Meaningless	“N/A”, “None”, “None, Other”, “0”	168	
	HOA Amount = No Info	Missing	“0”, “Unknown”, “See Remarks”, “Other”, null	30279	
		Valid	“None – See Remarks”, “None”, “No Association Fee”, “N/A”	11	
HOA Features	HOA Amount >0	Missing	“Unknown”, “See Remarks”, “See Remarks, Other”, “Other Amenities”, “Other”, null	14989	
		Meaningless	“None”, “No Amenities", “None, See Remarks”, “None, Other”	8	
	HOA Amount = No Info	Missing	“Unknown”, “Other”, null	30463	
		Valid	“None”, “No Amenities”	5	
HOA Frequency	HOA Amount >0	Missing	“See Remarks”, “Voluntary”, null, “Included In Property Tax”, null	2493	
		Meaningless	“Not Applicable”, “None”	1005	
		Missing	“See Remarks”, null, “Included In Property Tax”,	32484	

	HOA Amount = No Info	Valid	“N/A”, “None”, “Not Applicable”	998
Cooling type	Cooling Y/N – Yes	Missing	“Insert”, “Insert, Other – See Remarks”, null, “Other”, “Other – See Remarks”, “Other, See Remarks”, “See Remarks”, “See Remarks, Other”, “Total”, “No Air Conditioning”	494
		Meaningless	“N/K”, “No cooling”, “No cooling, Other”,	4
	Cooling Y/N – No Info	Missing	Null	10146
	Cooling Y/N – No	Valid	“None”, “None, Other”, “None, See Remarks”	5
Heating type	Heating Y/N – Yes	Missing	“Insert”, ”Other, See Remarks”, “Other”, “Other – See Remarks”, “See Remarks”, “See Remarks, Other”	857
		Meaningless	“No Heat”, “No Heat, Other”, “No Heating”	14
	Heating Y/N – No Info	Missing	Null	7279
	Heating Y/N - No	Valid	“None”, “None, Other”, “None, See Remarks”, “Other, None”	5

Table 9 Missing Values Part (author)

3.4 Data Preparation

The aim of the Data Preparation phase is to ensure that the dataset is clean, consistent, and suitable for further analysis and modelling. Working with unclean or inconsistent data can lead to misleading results, unreliable insights, and poorly performing models.

In this study, the data preparation process is guided by the attribute-oriented approach, where each attribute is examined and processed individually or in small groups based on their similarity or interdependence. This method enables a focused treatment of issues such as missing values, inconsistencies, and formatting irregularities. By addressing each attribute according to its specific characteristics and role within the dataset, the approach ensures a higher level of data quality and relevance for subsequent analysis. In cases where attributes are conceptually or structurally related, they are preprocessed together to maintain internal coherence.

3.4.1 Dataset-wide transformations

Duplicates

In the previous step of Data Understanding 11 773 listings were marked as duplicates. From each group of attributes, only 1 was left.

Formatting

During the Data Understanding step, it was found out that most attributes, which should have data types integer or float, are presented as strings in the dataset. The reason is that lines contain values like "$", "—", "No Info". To work properly with them, we need to fix their data types and remove all interfering values. For making changes Microsoft Power Query Tool will be used. It is integrated into Microsoft Excel and is effective due to visualization.

Values "—" is a missing value in attributes "Bedrooms", Baths", "Full Baths", "Square Feet", "Acres", and "Year Built" was changed to null.

```
= Table.ReplaceValue(#"Záhlaví se zvýšenou úrovní","—
","",Replacer.ReplaceText,{"Bedrooms", "Baths", "Full Baths", "Square Feet", "Acres",
"Year Built"})
```

Code 14 Replacing "-" in certain attributes (author)

"$" sign appears in attributes "Price" and "Price/Sq Ft" was changed to null

```
= Table.ReplaceValue(#"Nahrazená hodnota","$","",Replacer.ReplaceText,{"Price", "Price/Sq
Ft"})
```

Code 15 Removing $ sign from certain attributes (author)

The chosen website uses "," separator to separate thousands and "." separator to separate the decimal part of the number. However, Excel uses different decimals depending on the localization of the Excel. The American version of Excel to separate the decimal part of the number uses ".", while in the Czech version of the program, the" separator is used to separate the decimal part of the number, while the "." separator is not used at all. Since version installed on my PC is Czech we have to process all "," and "."

```
= Table.ReplaceValue(#"Nahrazená hodnota1",",","",Replacer.ReplaceText,{"Square Feet",
"Price"})
= Table.ReplaceValue(#"Nahrazená hodnota2",".",",",Replacer.ReplaceText,{"Acres", "HOA
Amount"})
```

Code 16 Replacing separator in certain attributes (author)

"No Info" is a value representing missingness in attributes "Cooling Y/N", "Heating Y/N", "HOA Amount", "Price/Sq Ft". While "HOA Amount" and "Price/Sq Ft" will be integer, "Cooling Y/N" and "Heating Y/N" will be Boolean attributes. In both cases, "No Info" must be read by both Excel and Pandas as a null value.

```
= Table.ReplaceValue(#"Nahrazená hodnota3","No Info","",Replacer.ReplaceText,{"Price/Sq Ft", "HOA Amount", "Cooling Y/N", "Heating Y/N"})
```
Code 17 Replacing "No Info" in certain attributes (author)

After the removal of values that prevent attributes from being read in the needed data types, they can be transformed into an integer or a float, depending on the context.

```
= Table.TransformColumnTypes(#"Filtrované řádky",{{"Bedrooms", Int64.Type}, {"Baths", Int64.Type}, {"Full Baths", Int64.Type}, {"Square Feet", Int64.Type}, {"Year Built", Int64.Type}, {"Price", Int64.Type}, {"Price/Sq Ft", Int64.Type}, {"HOA Amount", Int64.Type}, {"Garage Spaces", Int64.Type}, {"Parking Total", Int64.Type}, {"Units Total", Int64.Type}, {"Acres", type number}, {"Agency Fee", type number}})
```
Code 18 Changing data types(author)

"Cooling Y/N" and "Heating Y/N" "Yes/No" values will be changed to 1/0 respectively.

```
= Table.ReplaceValue(#"Změněný typ","Yes",1,Replacer.ReplaceValue,{"Cooling Y/N", "Heating Y/N"})
= Table.ReplaceValue(#"Nahrazená hodnota5","No",0,Replacer.ReplaceValue,{"Cooling Y/N", "Heating Y/N"})
```
Code 19 Replacing "Yes/No" values in certain attributes(author)

Attributes Property Type and Property Sub Type

"Property Type" is a key attribute in this analysis, as it defines the category of the property listed (e.g., Residential, Land, Farm, etc.). Different property types serve different purposes and exhibit distinct characteristics, which makes direct comparison between them inappropriate. All listings with values of the attribute "Property Type" different from "Residential" and "Single-family residence / Townhouse" would be removed from the dataset. Property type "Single family residence / Townhouse" consists only of 2 listings, and the value of the attribute will be changed to "Residential".

Another important attribute is "Property Sub Type," which specifies the specific type of residential real estate. All values in this attribute are relevant to the scope of this study, except for "Boat Dock" and null. Although these listings are technically categorized under residential property, they cannot be considered as habitable real estate. Furthermore, upon manual verification, it was found that some of these listings are actually vacant land rather than docks, and 2 out of 15 listings are condominiums and will be left. These listings are different from all others in this "Property Sub Type" group, since they contain values in columns "Bedrooms", "Baths", "HOA Frequency", and interior and exterior features. The other 15 will be excluded from the data

3.4.2 Identifying and removing data outside an acceptable range of values

This step was designed to identify and remove values that, while technically valid (e.g., non-null and numeric), fall outside a logically or contextually acceptable range. These entries — such as "Square Feet" = 1, "Price" = 0, or "Year Built" = 1200 — do not reflect realistic real estate listings and could distort statistical summaries and imputation methods. Performing this cleansing prior to missing value handling is crucial, as it ensures that derived statistics (e.g., medians, IQRs) are calculated on meaningful data. Therefore, such values were removed before the imputation stage to preserve statistical integrity.

Attributes Price, Square Feet, Price/Sq Ft

To remove bottom values, we will use the business understanding of the real estate market. For the attribute "Price," 2 bottom filters and 1 upper filter were used. For listings, where price < 15,000$ and "Bedrooms" != "0", only the attribute "Price" will be cleared, since price like this most likely is a mortgage payment or monthly rent. Houses with prices between 15 000$ and 45,000 can be damaged houses or ones which need significant investments on renovation, or can be used in a flip business, or elite houses with extremely high rent prices. This group of houses will be removed from the dataset; the total number of these listings is 469.

To set up the upper filter, we will build a spotter plot and check percentiles. Properties with a price above 5 000 000$ can be considered as ultra luxurious real estate, which is far from the average house and represents a separate segment of the market. Since research is being held to figure out general trends, houses with such a high price will be excluded. At this step, we can build a graph and notice in which cities elite objects are located. In the .xslx file, Boston listings occupy lines from 3295 to 4425, Los Angeles and Miami from 20029 to 22663 and 24548 to 29615, respectively, which is noticeable on the graph.

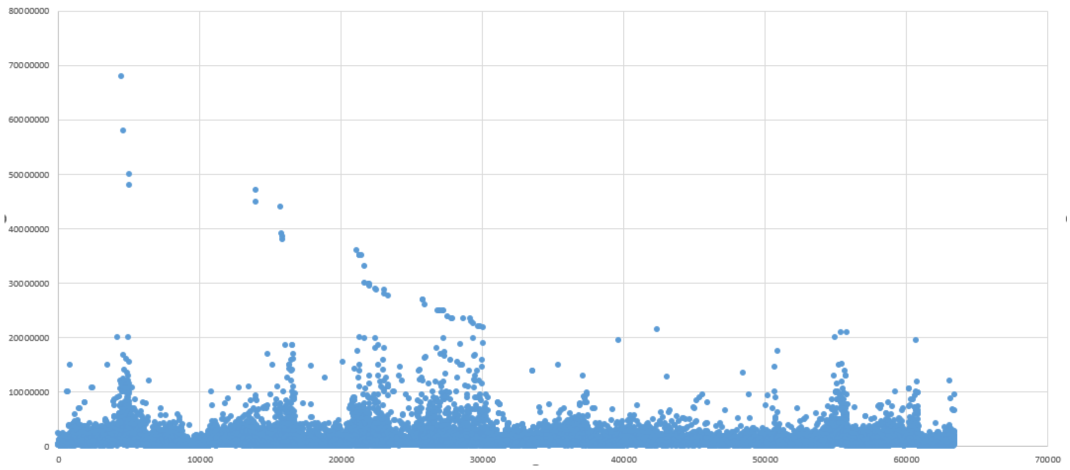

Figure 15 Price spotter plot. X - Listing ID, Y – Price (author)

Price	Number of listings
$1m <= Price < $2m	6851
$2m <= Price < $3m	1813
$3m <= Price < $5m	1095
$5m <= Price < - $10m	516
Price >= $10m+	206

Code 20 Number of listings within in certain price range (author)

As we can see from the table, houses with a price from $1m to $5m have a significant percentage of the dataset, with a total number of 17,11%, basically, each 6th listing. Moreover, 65,36% of these listings are property objects in the lower price range of $ 1 m-$2 m. Based on these facts, we can make a conclusion that listing above $2m can be considered not as upper segments, but as luxurious segments, while listing above $5m – as ultra luxurious. In support of these assumptions, median values of the "Square feet" attribute were counted, and the following results were obtained. Listings between $2m and $5m will be marked as "luxurious" in the dataset, while listings with a price of $5m or more will be deleted from the dataset. The total number of removed listings is 722.

Price	**Median square**
>=2 000 000	3670
>5 000 000	5286
< 2 000 000	1540

Table 10 Median square of listings in a certain price range (author)

As we can see below, there are 11 listings where "Square Feet" < 150; Square feet will be recalculated.

The upper border was set based on the scatter plot for listings, where "Is Luxurious" = "Yes". Values above 12,000 square feet will be considered outliers.

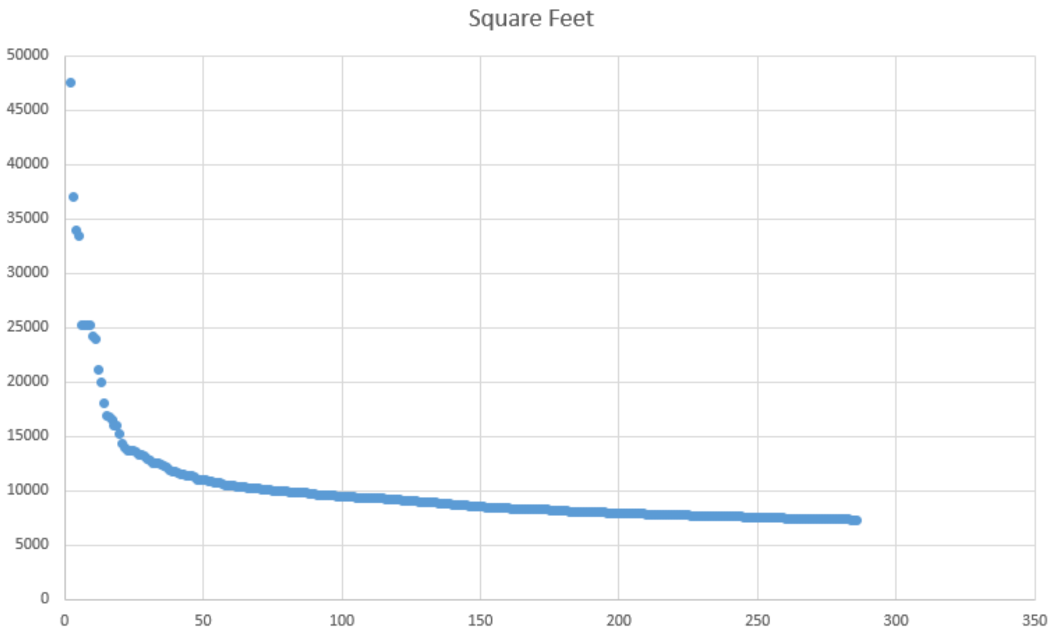

Figure 16 Median square feet spotter plot. X-number of listings, Y - median square feet (author)

To set up a filter for "Price/Sq Ft" attribute value, $40 was chosen as the bottom. From 110 listings, 15 listings cost below $100 000 which most likely show values, needed serious investing. Regarding the upper limit value of $2000, it contains less than 0.5% listings in the dataset, and 70% of these listings belong to luxury real estate, while this group of listings is relatively small. Only 2 outlier listings were cut, since they contain values too high compared to all others, with values of 4250 and 6188.

Bedrooms, Baths, Full Baths

91.4% of all objects – objects with 2-4 bedrooms, which is relatively logical for the market of the United States. Listings with more than 6 bedrooms are only 0.75% of the whole dataset. According to the IQR method, values above 8 bedrooms were defined as outliers. However, since in our dataset listings from the premium segment appear too, we will define separate filters. For objects "Is Luxury" = "No" filter of "Bedrooms" < 8 is applied. For "Is Luxury" = "Yes" < 12 filter is applied.

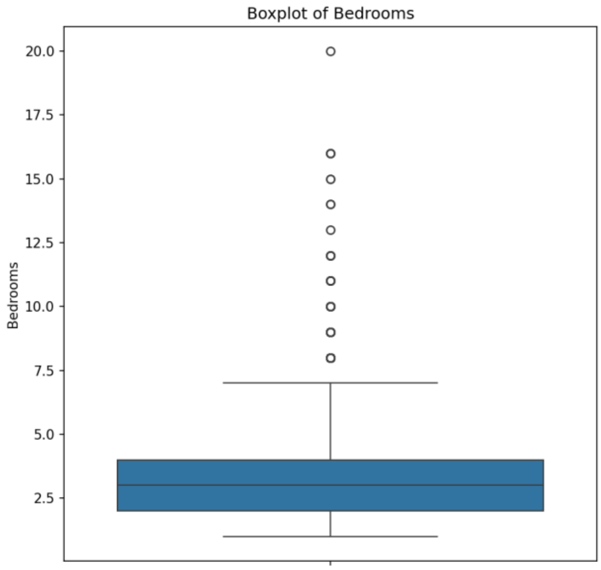

Figure 17 Boxplot of Bedrooms (author)

Because property objects usually don't contain a lot of bathrooms and only one may be in the house for a few bedrooms, the IQR method is significantly shifted down. For this reason, we will use the same filter for the "Full Baths" attribute for no-luxury objects. In the "Baths" attribute, outliers are above 10 .

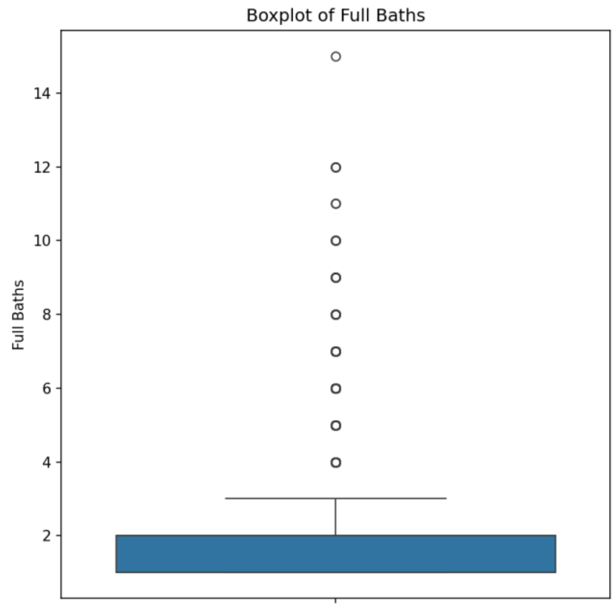

Figure 18 Boxplot of Full Baths (author)

Year Built

Since the oldest city in the U.S. was founded in 1565, values below this value will be cleared and recounted. The highest realistic value is 2026, since the values after this is 9999.

Acres & Property Sub Type

Attribute “Acres”, which must represent the size of the lot is hard for interpretation, because dataset contain different types of property(attribute “Property Sub Type”), including those, which should not have its own land, for example, condominiums and condo/townhouses, where land is owned by HOA or objects, where most of the land is occupied by building and might have only little backyard: townhouses and duplexes. However, these listings contain “Area” values. 4531 out of 15022 condominiums have values different from “null” or “0”. In this case, it is hard to interpret the meaning of these values. They might represent either size of the common facilities(GYM, Swimming pool, working areas) which are often present in condominiums. For standardization, the attribute “Acres” for all listings where “Property Sub Type” = “Condominium”, “Condo/Townhouse”, or “Townhouse” was changed to null.

Manufactured homes and mobile homes are usually located on their own land and represent the same subtype of house. Mobile homes are factory-built homes with permanently attached chassis, which allows them to be transported to the site. (“Mobile Home,” 2025) In 1976 United States Department of Housing and Urban Development set standards for this type of building, upgrading standards and setting the foundation. Among changes were fire safety, plumbing and electrical rules, and frame norms. Basically, subtypes “Manufactured on Land”, “Manufactured Home”, and “Mobile Home” represent the same concept and will be changed to “Manufactured”. Moreover, in our dataset, mobile homes have a median “Year Built” of 1993, which suggests that listing owners might get confused between 2 meanings. Property Sub Types of “Duplex” and “Timeshare” were removed from the dataset due to an insufficient number of listings representing these types of properties.

For all listings that have been left in the dataset, the next methodology will change all values higher than 5 to null and recount. To define bottom values, we will look at the square of the house. It is logical, that size of the lot can't be smaller than the size of the house. If it happens, most likely area, which is free from buildings was put. These values were changed for null.

```
#Creating mask for properties where we need to clean Acres
target_types = ["Single Family Residence", "Manufactured"]
mask_target_types = df["Property Sub Type"].isin(target_types)

#Creating a valid mask where both Square Feet and Acres are available
mask_valid = mask_target_types & df["Square Feet"].notna() & df["Acres"].notna()

#Looking for listings where lot size is smaller than building size
mask_invalid_acres = mask_valid & (df.loc[mask_valid, "Acres"] * 43560 <
df.loc[mask_valid, "Square Feet"])

#Replacing invalid Acres with NaN
df.loc[mask_invalid_acres, "Acres"] = pd.NA
```

Code 21 Cleaning invalid lot size data (Acres) for specific property types (author)

Garage Spaces, Parking Total

In both attributes, there are extremely high values. For single-family residences, it might be a mistake, while for condominiums, the listing owner might put a total number of parking spaces, belonging to the building.

Percentile analysis shows that 99% of listings have fewer than 8 parking spaces and 4 garages. These values will be left as the upper borderю

3.4.3 Missing values

Before starting to fill in missing values, I have decided to double-check the dataset for duplicates. It was noticed that some listings are similar to each other, since they are located in the same location by "Location Request" and "Zip-Code", contain the same number of bedrooms, baths, and have the same square footage. The problem might appear because the object was posted a few times by different agents or owners with different prices. The same algorithm was used to remove duplicates from the dataset.

Square feet, Price, Price/Sq Ft, Bedrooms, Baths, Full Baths, Acres

After removing all outliers, we can fill in missing values using the medians of the specific groups. It means we won't fill these values just by the median of the whole column, but we will create groups(bins) based on attributes "Location Request", "Property Sub Type", and already filled needed attributes. For example, to fill the missing value in square feet, we will look for the most similar listings and use their median for filling.

Attribute	Number of missing values	Grouping by
Square Feet	2884	“Location Request”, “Property Sub Type”, “Price Bin”
Price	1152	“Square Feet Bin”, “Location Request”, “Property Sub Type”
Price/Sq Ft	3995	-
Bedrooms	766	“Square Feet Bin”, “Location Request”, “Property Sub Type”
Baths	143	“Bedrooms”, “Location Request”, “Property Sub Type”
Full Baths	681	“Location Request, Property Sub Type”, “Full Baths”
Acres (where “Property Sub Type” = “Manufactured”, “Single Family Residence”	5008	“Location Request”, “Property Sub Type”, “Square Feet”

Code 22 Number of missing values for each attribute (author)

As can be seen from the table, to fill values “Square feet” and “Price,” we need to have one of these attributes already filled. 31 listings have missing values in both attributes, and they were removed.

8 listings where “Square feet” = null has filled “Price” and “Price Sq/Ft” and they can be counted based on these attributes. It happened, because initially these values were written as decimal numbers with dot-separator. After changing data type of “Square Feet” from string to integer in Microsoft Power Query these values were taken as mistake and removed by the program. On this step they were filled and checked for passing the requirements described in the previous paragraph 3.4.

To fill missing values in the attribute “Square Feet” next algorithm was used:

```
# Creating Square Feet Bin (binning Square Feet by 300 sqft intervals)
df["Price Bin"] = (df["Price"] // 100000) * 100000

#Calculating the median Square Feet grouped by Location Request, Property Sub Type,
Bedrooms, and Price Bin
group_sqft = df.groupby(["Location Request", "Property Sub Type", "Price Bin"])["Square
Feet"].median()

#Defining a function to fill missing Square Feet values
def fill_square_feet(row):
    if pd.isna(row["Square Feet"]):
        key = (row["Location Request"], row["Property Sub Type"], row["Price Bin"])
        if key in group_sqft:
            return group_sqft[key]
```

```
    return row["Square Feet"]

#Applying the function to fill missing Square Feet
df["Square Feet"] = df.apply(fill_square_feet, axis=1)
```
Code 23 Imputing missing Square Feet values (author)

Because attributes “Price” and “Square Feet” contain a large number of unique values, they must be combined into smaller groups(bins). Interval for binning “Square Feet” was set on 300 sq ft and was used to fill missing values in the attribute “Price”, while for “Price” – 100,000$ and vice versa was used to fill missing values in the attribute “Square Feet”

```
# Creating Square Feet Bin (binning Square Feet by 300 sqft intervals)
df["Square Feet Bin"] = (df["Square Feet"] // 300) * 300

# Calculating median Price grouped by Square Feet Bin, Property Sub Type, and Location
Request
group_price = df.groupby(["Square Feet Bin", "Property Sub Type", "Location
Request"])["Price"].median()

# Defining a function to fill missing Price
def fill_price(row):
    if pd.isna(row["Price"]):
        key = (row["Square Feet Bin"], row["Property Sub Type"], row["Location Request"])
        if key in group_price:
            return group_price[key]
    return row["Price"]

# Applying the function
df["Price"] = df.apply(fill_price, axis=1)
```
Code 24 Imputing missing Price values (author)

A similar algorithm was used to fill the attributes “Bedrooms”, “Baths”, “Full Baths”, and “Acres” attributes. However, 19 values in the attribute “Square Feet”, 46 in the attribute “Price”, and 70 in the attribute “Bedrooms” were left as null, because of the absence of a group based on the chosen attributes. The number of listings is insufficient and will be removed from the dataset.

The attribute “Acres” contained the largest amount of missing values, and after filling with the median values of listings grouped by chosen attributes, it still contained 339 missing values. Remaining values were filled by the median of listings grouped just by one attribute – “Location Request”.

Categorial attributes

There are 4 attributes in our dataset responsible for Cooling and Heating. One is binary and says if there is cooling or heating in the property object, while the second explains the type. In “Cooling Type” and “Heating Type” attributes, features are separated by a comma.

Both Cooling Y/N and Heating Y/N contain a sufficient number of missing values. For all values where Cooling/Heating Y/N = null, all Cooling/Heating Type = null, either(except 1 listing which contained “Cooling Type” and 2 listings, which contained “Heating Type”. All Cooling/Heating Type, where Cooling/Heating Y/N = null, were changed to “0”.

However, some listings, which initially had “Cooling Y/N” or “Heating Y/N” filled with “0” still contain valid features in “Cooling Type”, “Heating Type” respectively. 62 values in “Cooling Y/N” were changed to “1” and 38 values in “Heating Y/N”.

As in attributes “Cooling Type”, “Heating Type” values in attributes “Interior Features”, “Exterior Features” are separated by a comma. Missing and meaningless values described in Table 7 will be changed to null.

HOA Attributes

I have decided to carry out work on this group of attributes separately, step-by-step, since they are interconnected, contain both numerical and categorial attributes, and need additional formatting and standardization.

Formatting

One of the significant issues with the HOA attributes is the lack of standardization. The reason is that some objects require paying the HOA fee monthly, while others annually, once every 3 or 6 months. To be able to compare HOA attributes between themselves, “HOA Amount” for all listings must be filled out for the same frequency. I have decided to use the month as a standard, and “HOA Amount” for listings with any other “HOA Frequency” will be recounted.

```
# Normalizing frequency values to lowercase
df["HOA Frequency"] = df["HOA Frequency"].str.lower()

# Defining conversion ratios for each frequency to monthly
frequency_to_months = {
    "annually": 12,
    "annual": 12,
    "yearly": 12,
    "semi-annually": 6,
    "quarterly": 3,
    "monthly": 1  # Already monthly, no change needed
}

# Applying conversion
for freq, divisor in frequency_to_months.items():
    mask = df["HOA Frequency"] == freq
```

```
    df.loc[mask, "HOA Amount"] = df.loc[mask, "HOA Amount"] / divisor
    df.loc[mask, "HOA Frequency"] = "monthly"
```
Code 25 Standardizing HOA Amount to monthly values (author)

Identifying and removing data outside an acceptable range of values

After this outlier values must be removed. Although extremely low HOA fees may initially appear as outliers, they can be common in certain residential communities, especially if community consist of single home residencies, which include much less services and maintenance comparing to condos, where HOA is responsible for maintaining of the building itself(repairing elevators, roof, etc.) and common areas, which are very popular in that type of accommodation(doorman, GYM, swimming pool and so on). The bar chart below compares the overall distribution of listings by Property Sub Type with their representation in the bottom 5% of HOA Amount values.

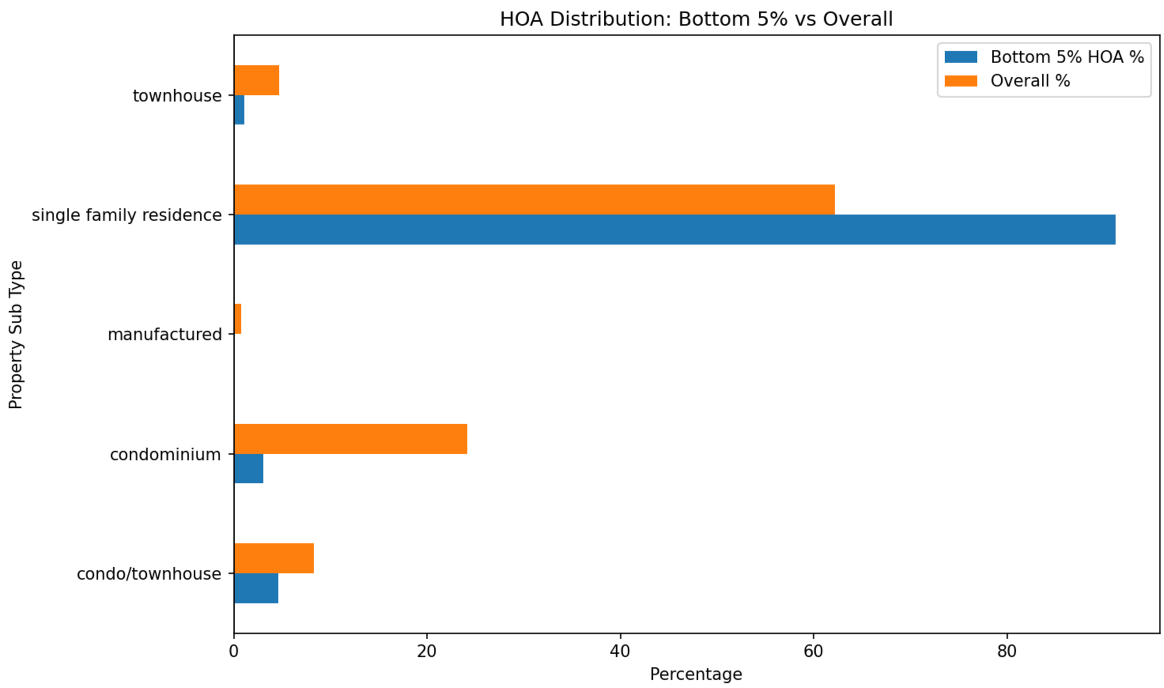

Figure 19 HOA Distribution (author)

Because of this reason, minimal value for different types of properties was set up differently: $1 for “Single Family Residence”, “Manufactured”, “Townhouse”, $30 for “Condominium”, $10 for “Condo/Townhouse”. A total number of values below the set is less than 60, and they will be removed and replaced.

To set up the upper border, an HOA/Price attribute was created, which represents the division of the HOA by the total price of the property. The problem may not be in the high numbers of HOA amounts per month, but in high numbers relative to the price of the object itself. For example, the situation where the HOA is 2000$ per month for the apartments whose price is 100 000$ is impossible. To solve this problem, attribute “HOA/Price” and it will help to set up the upper and lower. It is important to note that HOA/Price values are dependent on the attribute “Property Sub Type”, since single-family residences have relatively low HOA fees compared to condominiums, but might cost more. HOA/Price > 0.01 can be considered as outlier values, and 265 values will be removed as well.

Missing values

Attributes “HOA Includes”, “HOA Features”, “HOA Frequency” are similar to attributes “Cooling Type”, “Heating Type”, “Interior Features”, “Exterior Features”.

First of all, we have to take care of Valid values described in paragraph 3.4, and they will be changed to “Not Apply”.

```
valid_hoafreq = ["N/A", "None", "Not Applicable"]
valid_hoafeat = ["None", "No Amenities"]
valid_hoaincl = ["None - See Remarks", "None", "No Association Fee", "N/A"]
# Function to replace valid values with "Not Apply" if HOA Amount is missing
def replace_if_null_hoa(value, valid_list, hoa_amount):
    if pd.isna(hoa_amount) and isinstance(value, str) and value.strip() in valid_list:
        return "Not Apply"
    return value
df['HOA Frequency'] = df.apply(
    lambda row: replace_if_null_hoa(row['HOA Frequency'], valid_hoafreq, row['HOA
Amount']),
    axis=1
)

# Applying transformation to HOA Features
df['HOA Features'] = df.apply(
    lambda row: replace_if_null_hoa(row['HOA Features'], valid_hoafeat, row['HOA
Amount']),
    axis=1
)

# Applying transformation to HOA Includes
df['HOA Includes'] = df.apply(
    lambda row: replace_if_null_hoa(row['HOA Includes'], valid_hoaincl, row['HOA
Amount']),
    axis=1
)
```

Code 26 Reclassifying valid HOA-related values as “Not Apply” (author)

All missing and meaningless values from the table will be changed to “null”.

Some listings have mentioned “HOA Amount”, but have missing values in the attribute “HOA Frequency”. The simplest way could be filling these missing values with “monthly”, since it is the most popular frequency; however, in our initial dataset, there were listings with annual, semi-annual, and quarterly values. To solve this problem, the 3rd quantile was counted for each property subtype, and then multiplied by the number of months.

	50th Percentile	75th Percentile
Property Sub Type		
Condo/Townhouse	0.000756	0.001259
Condominium	0.001214	0.001867
Manufactured	0.000429	0.006488
Single Family Residence	0.000106	0.000211
Townhouse	0.000500	0.000761

Figure 20 OA/Price Distribution by Property Sub Type (50th and 75th Percentiles) (author)

```
def assign_frequency(row):
    subtype = row["Property Sub Type"]
    if subtype not in thresholds:
        return np.nan
    if pd.isna(row["HOA Amount"]) or pd.isna(row["Price"]) or row["Price"] == 0:
        return np.nan

    ratio = row["HOA/Price"]
    base = thresholds[subtype]

    distances = {
        "Monthly": abs(ratio - base),
        "Quarterly": abs(ratio - base * 3),
        "Semi-annually": abs(ratio - base * 6),
        "Annually": abs(ratio - base * 12)
    }

    return min(distances, key=distances.get)

# Apply the logic only to rows with missing HOA Frequency
mask = df["HOA Frequency"].isna()
df.loc[mask, "HOA Frequency"] = df[mask].apply(assign_frequency, axis=1)
```

Code 27 Inferring missing HOA Frequency (author)

After executing this code, all “HOA Frequency” values were standardized and got the value “Monthly”

To fill missing values in the attribute “HOA Amount”, the chosen method was to calculate the median HOA-to-Price ratio (HOA/Price) for each group defined by Property Sub Type and Location Request. The missing HOA Amount was then estimated using the formula:

Estimated HOA Amount = Property Price × median(HOA/Price)

This approach allows the HOA fee to scale proportionally with the property's market value, reflecting the observed relationship between HOA costs and property prices within each group. It is particularly effective when the dataset contains a wide range of property prices in the same group, where fixed-value imputation would be less accurate.

```
#Counting median value
thresholds_df = df[df['HOA Frequency'] == 'Monthly'].groupby(
    ['Property Sub Type', 'Location Request']
)['HOA/Price'].quantile(0.5).reset_index()

# Creating lookup dictionary
thresholds_dict = {
    (row['Property Sub Type'], row['Location Request']): row['HOA/Price']
    for _, row in thresholds_df.iterrows()
}

#Function to recover HOA Amount if any HOA-related field is present
def recover_hoa_amount(row):
    if pd.notna(row['HOA Amount']):
        return row['HOA Amount']

    freq = row.get('HOA Frequency')
    incl = row.get('HOA Includes')
    feat = row.get('HOA Features')

    # If all HOA-related fields are missing or marked as "Not Apply" → skip
    if all([
        pd.isna(freq) or freq == "Not Apply",
        pd.isna(incl) or incl == "Not Apply",
        pd.isna(feat) or feat == "Not Apply"
    ]):
        return row['HOA Amount']

    # Proceed only if we have a threshold and a valid price
    key = (row['Property Sub Type'], row['Location Request'])
    if key not in thresholds_dict or pd.isna(row['Price']) or row['Price'] == 0:
        return row['HOA Amount']

    if pd.isna(freq) or freq == "Not Apply":
        row['HOA Frequency'] = "Monthly"

    estimated = row['Price'] * thresholds_dict[key]
    return round(estimated)

#Applying recovery function
```

```
df = df.apply(lambda row: row.assign(
    **{'HOA Amount': recover_hoa_amount(row)}
) if pd.isna(row['HOA Amount']) else row, axis=1)
```
Code 28 Recovering missing HOA Amount (author)

Year Built

To fill missing values in the attribute "Year Built," we will use the method of filling with the median by group. Chosen attributes will be "Property Sub Type", "Location Request"

3.5 Exploratory Data Analysis

The goal of Exploratory Data Analysis(EDA) is to find patterns and relationships that help explain the market in different locations, set up hypotheses on what drives differences in prices, and define main market trends.

Property Sub Type

Property Sub Type is one of the key metrics in our analysis, since it directly reflects the structural and functional characteristics of residential units and significantly influences pricing, being one of the foundations of segmentation. For these reasons, we have to take into account in the analysis.

The bar chart below shows the difference in the property subtypes between the chosen U.S. cities.

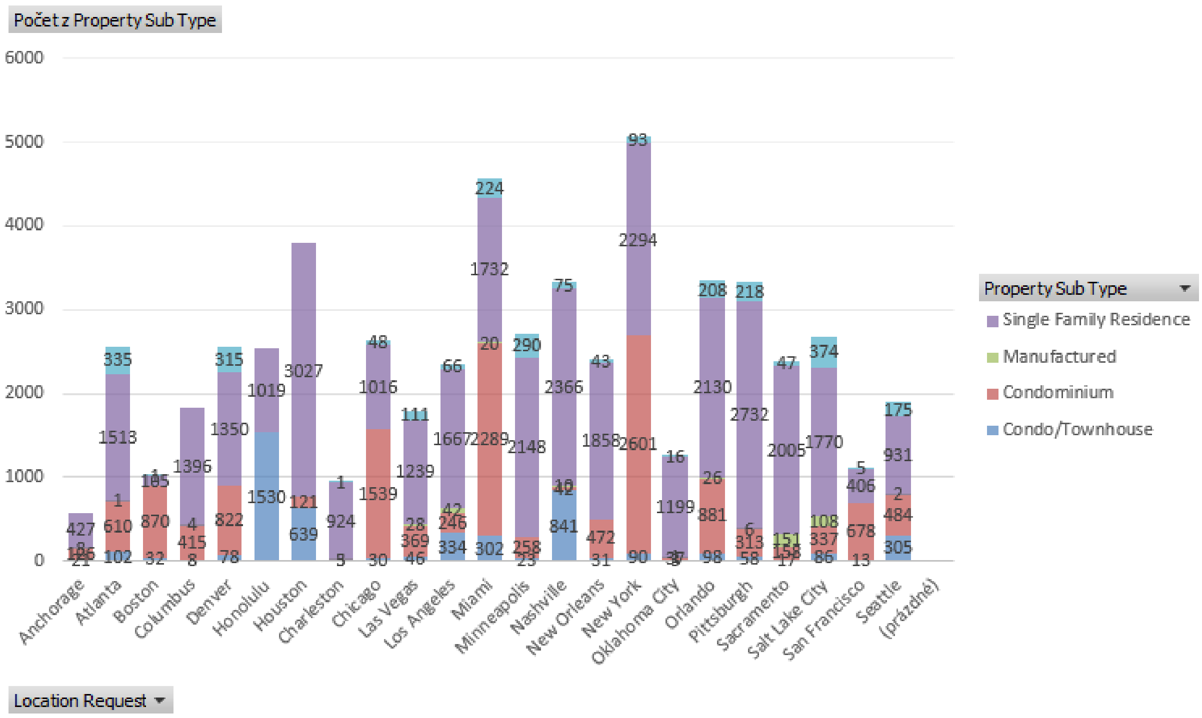

Figure 21 Number of listings of each property type for cities in the dataset (author)

As we can see in both metrics, manufactured houses have much lower values, and since they don't represent a significant part of the dataset, I have decided to exclude them from future analysis. Also, as we can see in Figure 21, some locations contain only a small number of listings with certain property types, and they will be removed, since they can't represent the tendency in the whole city.

Property Sub Type	**Location**	**Number of listings**
Condominium	Charleston	1
Townhouse	Charleston	1
	San Francisco	5

Table 11 Property Sub Type with low number of listings (author)

While the type of property, Manufactured, is represented by more than 400 listings, they are distributed unequally. Only 2 cities, Salt Lake City and Sacramento, include more than half of the listings with this property type. To check if these listings are similar to "Single Family Residence" objects, I have decided to compare their mean footage and price of houses where "Is Luxury" = "0".

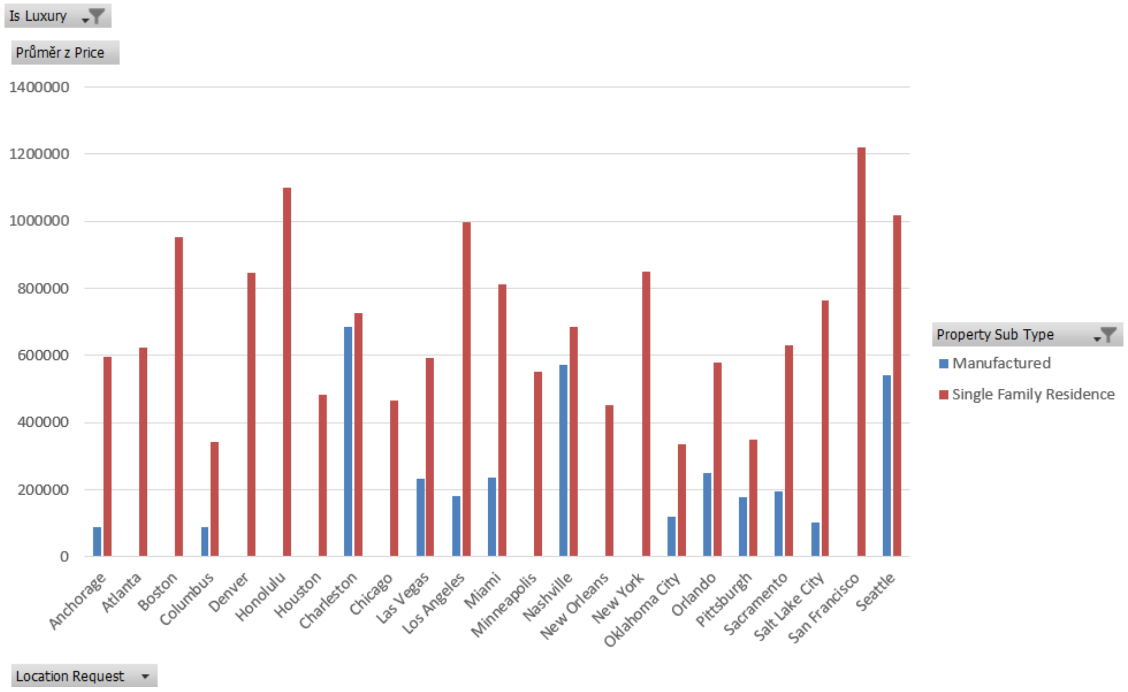

Figure 22 Comparing Manufactured/SFR properties by mean price (author)

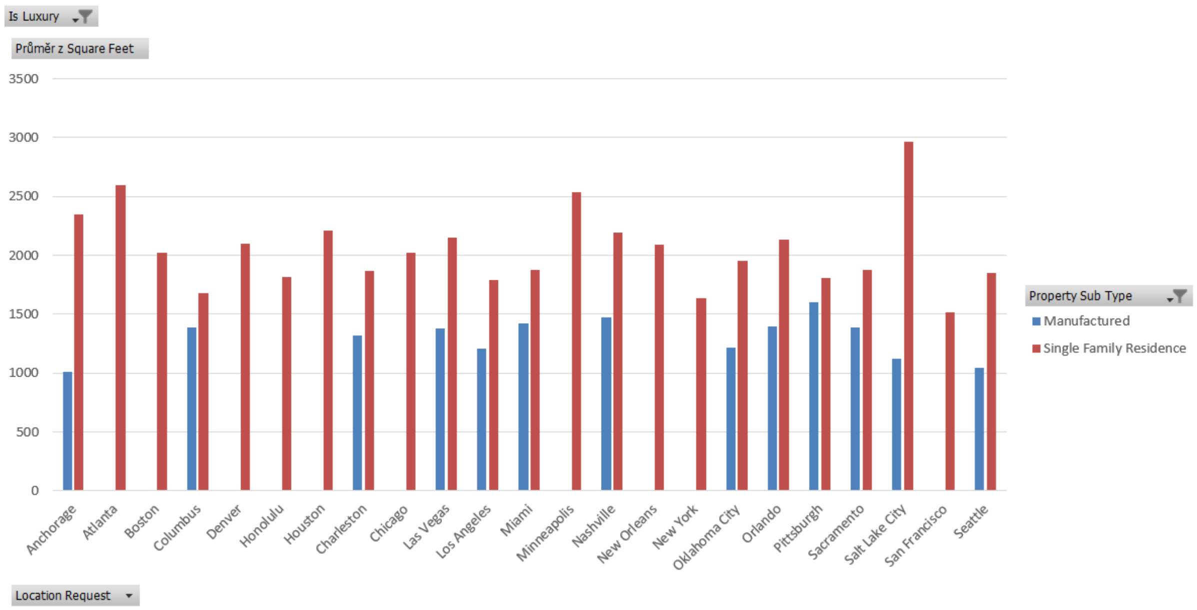

Figure 23 Comapring Manufactured/SFR properties by mean square feet (author)

As we can see in both metrics, manufactured houses have much lower values, and since they don’t represent a significant part of the dataset, I have decided to exclude them from future analysis.

It is visible on the chart that “Single Family Residence” dominates in 18 out of 23 analyzed cities.

However, notable exceptions include New York, Miami, Boston, and Chicago, where Condominiums represent the majority of listings. Additionally, Honolulu stands out with

Condo/Townhouse as the dominant type, while showing no listings labeled simply as "Condominium". This anomaly likely results from a local MLS classification issue, where townhouses and condos were merged into a single category.

To validate this assumption, the average square footage bar chart was created to compare the square footage of Condominiums, Condo/Townhouses, and Townhouses across various locations. For comparing subtypes in different locations, we have counted the mean values of the "Square Feet" attribute.

By logic, Condominium should be the smallest one, since the last 2 property subtypes represent townhouses, but with a different type of ownership. 20/23 cities confirm this assumption, and the difference in the other 3 is very small. As we can see, square feet of "Condo/Townhouse" properties in Honolulu is the lowest among all "Condo/Townhouse" properties and lower than all footages of Condominiums in all cities except New York. Because of these reasons, the "Condo/Townhouse" subtype in Honolulu will be changed to just "Condominium".

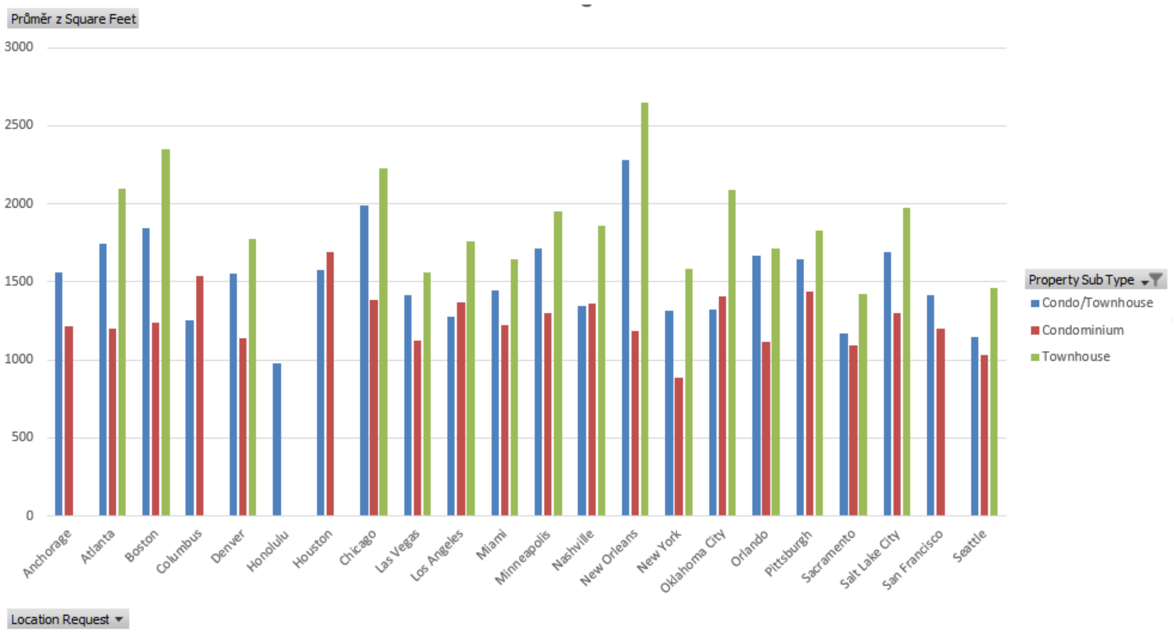

Figure 24 Comparing certain property subtypes by mean square feet (author)

Footage and Pricing

This section focuses on comparing the distribution of property size and price per square foot across different cities and property types.
To visualize and interpret market differences, we construct boxplots, which reveal the range, median, and outliers for each segment.

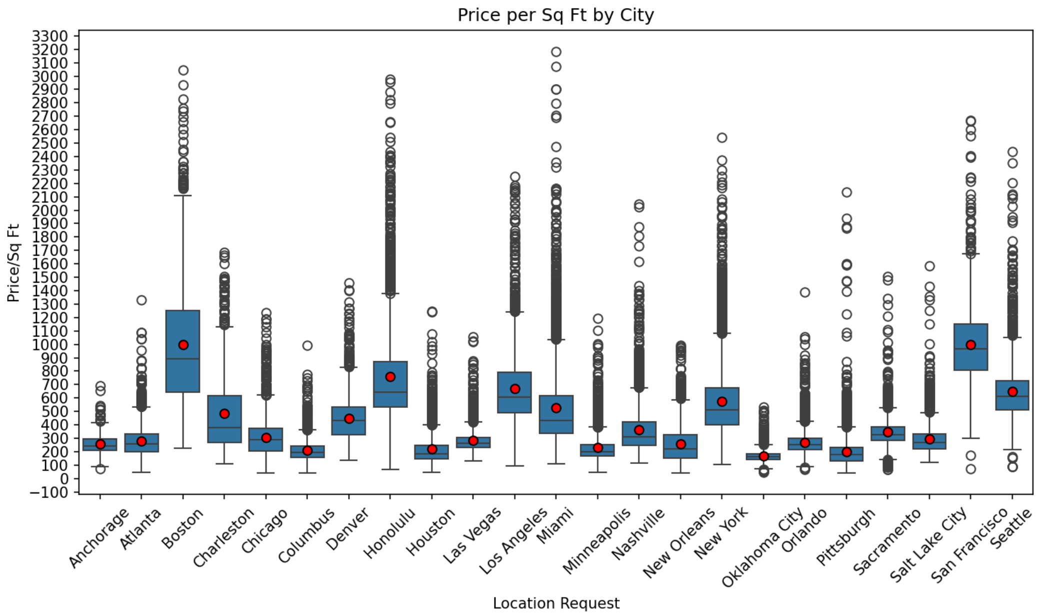

Figure 25 Price per Sq Ft by City Boxplots (author)

The boxplot of Price per Square Foot by city reveals substantial differences in both median values and market volatility. Coastal and high-density markets such as San Francisco, New York, and Boston show the highest median values, often exceeding $700/sqft, and are characterized by numerous high-end outliers.

In contrast, more affordable markets such as Columbus or Pittsburgh exhibit narrow interquartile ranges and lower overall prices.

Cities like Miami and Denver illustrate highly segmented markets, where luxury listings elevate the average significantly above the median, highlighting the importance of treating these markets with specialized modeling strategies.

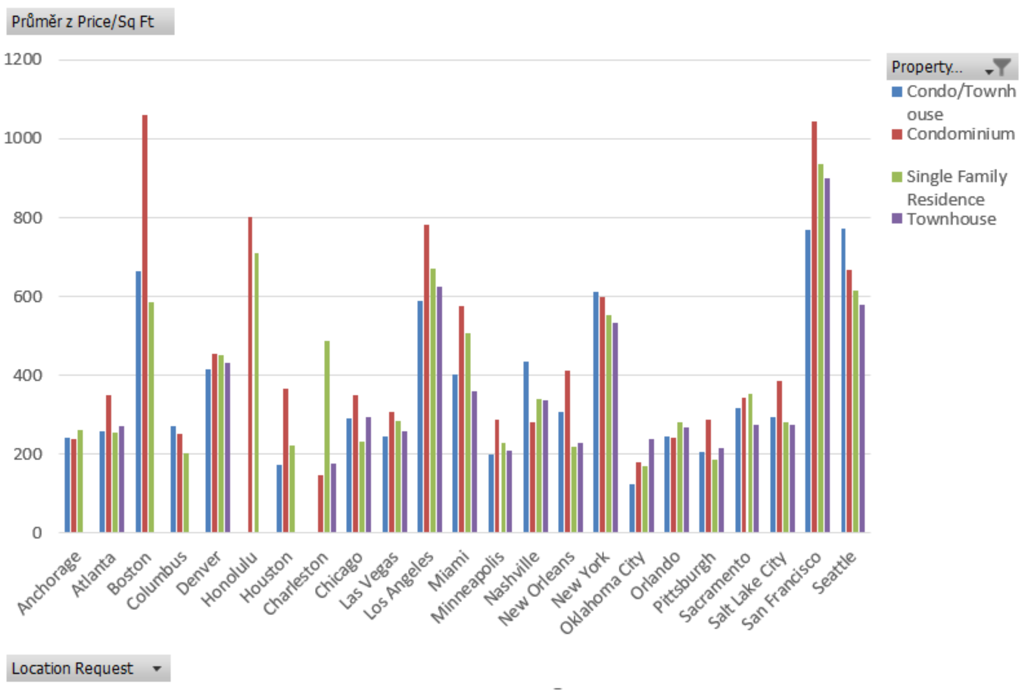

Figure 26 Comparing property subtypes in different locations by mean price/sq ft (author)

The highest average price/sq ft across all locations has the following property subtypes:

- Condominium – 14 locations
- Condo/Townhouse – 4 locations
- Single Family Residence – 4 locations
- Townhouse – 1 location

However, the difference between average price/sq ft is significant only in Boston and Houston, and due to probable lower footage, condominiums might be the most affordable type of housing to buy in most locations, even having the highest price/sq ft.

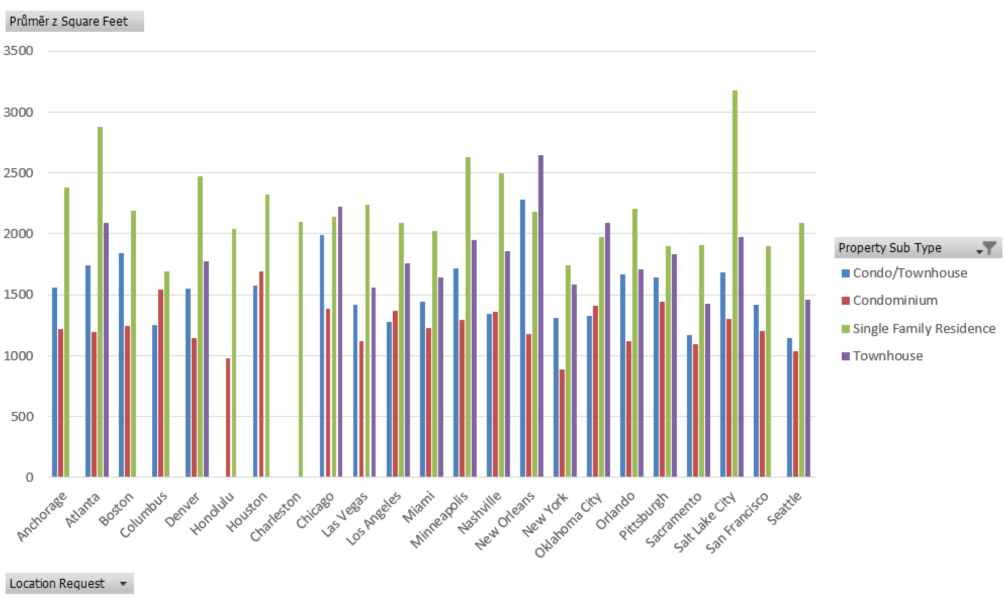

Figure 27 Comparing property subtype by mean square feet in different locations (author)

The most significant difference in footage between different types of properties in one city is in New York and Salt Lake City, where the average size of condominiums is 96.5% and 144.3% relatively lower than SFR.

The largest scatter between the same type of properties looks like this:

- SFR: Salt Lake City – 3176 sqft, Columbus 1693 sqft – 87.6%
- Condominium: Houston – 1693, New York – 887 – 90.9%
- Condo/Townhouse: Salt Lake City – 2176, Sacramento – 1424 – 52.8%
- Townhouse: New Orleans – 2283, Seattle – 1143, 99.7%

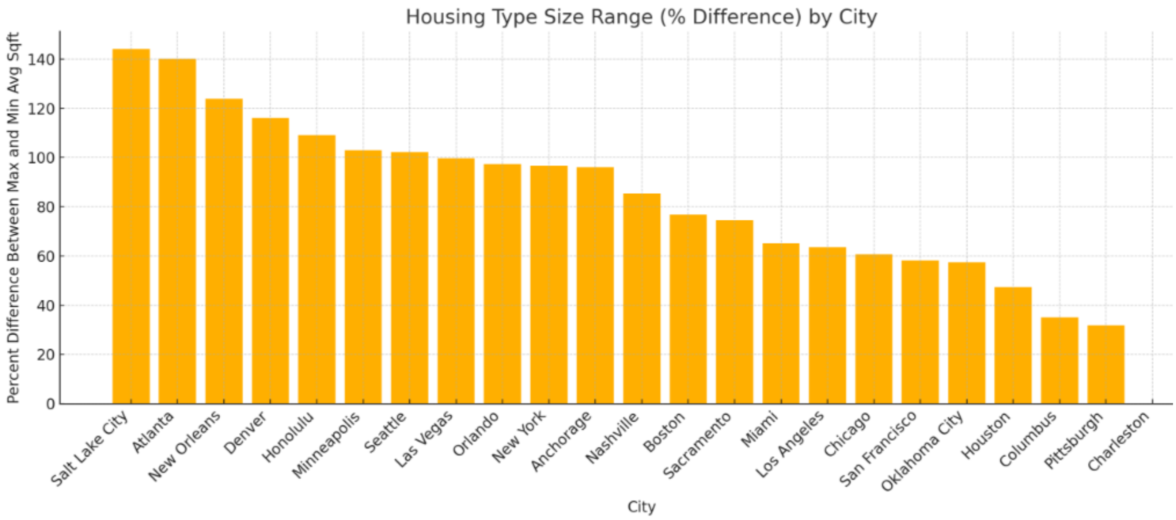

Figure 28 Housing Type Size Range by City (author)

Walk and drive rating

Another important assumption that can be made from the analysis of property subtypes across cities is logistics and infrastructure, which is presented in our dataset by attributes "Walk rating" and "Drive Rating". Infrastructure of Urban cities(where a high share of Condominiums) will be mostly made for pedestrians, and people will have to use public transport, which will increase the Walk rating, while the Drive rating will be low. Vice versa, infrastructure in Suburban must be more aimed at using one's own car(more roads, road junctions), while walk rating will be low.

To verify the assumption that different property types are associated with different levels of walkability and driving dependency, cities were grouped by their name, and the mean values of selected attributes were calculated:

```
by_city=df.groupby("LocationRequest").agg({
    "PropertySubType_Condominium":"mean",
    "PropertySubType_Townhouse":"mean",
    "PropertySubType_SingleFamilyResidence":"mean",
    "PropertySubType_Condo/Townhouse":"mean",
```

```
    "PropertySubType_Manufactured":"mean",
    "WalkRating":"mean",
    "DriveRating":"mean"
}).reset_index()
```
Code 29 Average property subtype proportions and accessibility ratings per city (author)

Next, all property subtypes were divided into two major categories:

- Urban housing: including Condominiums, Townhouses, and Condo/Townhouse hybrids.
- Suburban housing: Single Family Residences.

```
by_city["Urban_Housing_Share"] = (
    by_city["Property Sub Type_Condominium"] + by_city["Property Sub Type_Townhouse"] +
by_city["Property Sub Type_Condo/Townhouse"]
)

by_city["Suburban_Housing_Share"] = (
    by_city["Property Sub Type_Single Family Residence"] + by_city["Property Sub
Type_Manufactured"]
)
```
Code 30 Urban vs. Suburban housing share per city (author)

To visualize potential relationships, scatter plots were generated:

```
sns.scatterplot(data=by_city, x="Urban_Housing_Share", y="Walk Rating")
plt.title("Walk Rating vs Urban Housing Share")
plt.show()
```
Code 31 # Scatter plot: Urban housing share vs. Walk Rating (author)

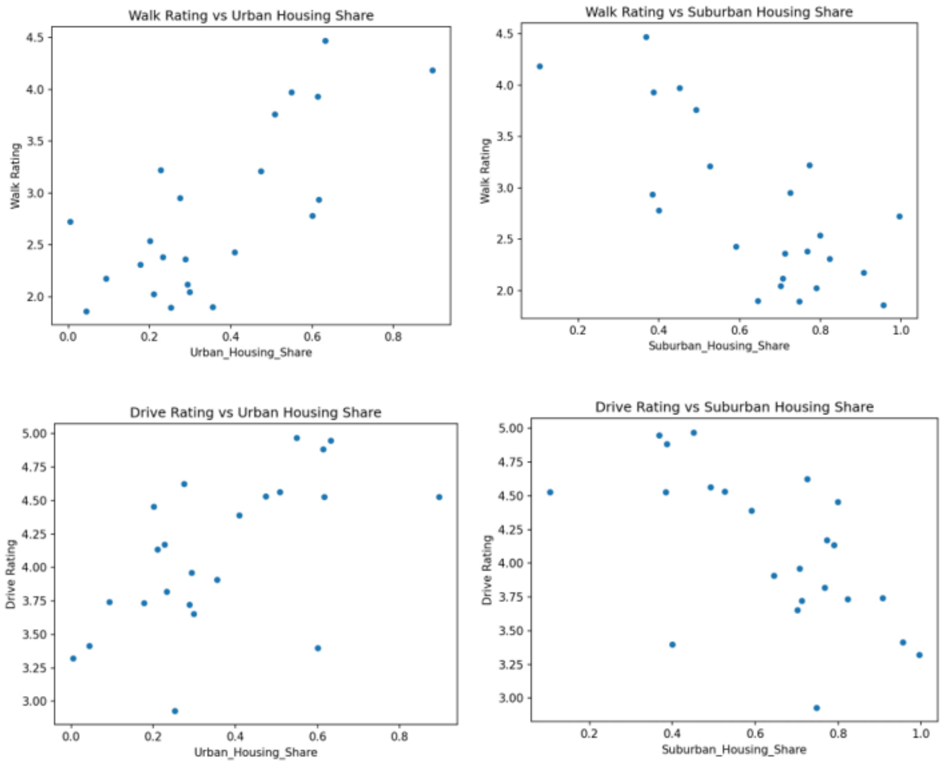

Figure 29 Urban vs. Suburban Housing Share and Accessibility Indicators (author)

Cities with a high share of urban housing types (e.g., condominiums and townhouses) tend to demonstrate higher scores both in Walk Rating and Drive Rating, suggesting greater mobility flexibility. This observation may reflect the fact that dense urban environments offer residents multiple viable transportation options — walking, driving, or using public transit — depending on context or preference. In contrast, suburban-dominated areas are more likely to show lower walkability, with car usage becoming a necessity rather than a choice. Moreover, heavy car dependence imposes infrastructural burdens — road expansion, parking space, and maintenance — which can reduce overall drive efficiency despite the car-centric layout.

HOA Attributes

HOA(Homeowners Association) is an association that runs a building or a district. Belonging to an HOA might be an important factor in choosing a real estate object, since an HOA can have a large number of restrictions or, vice versa, bonuses, and might have high fees.

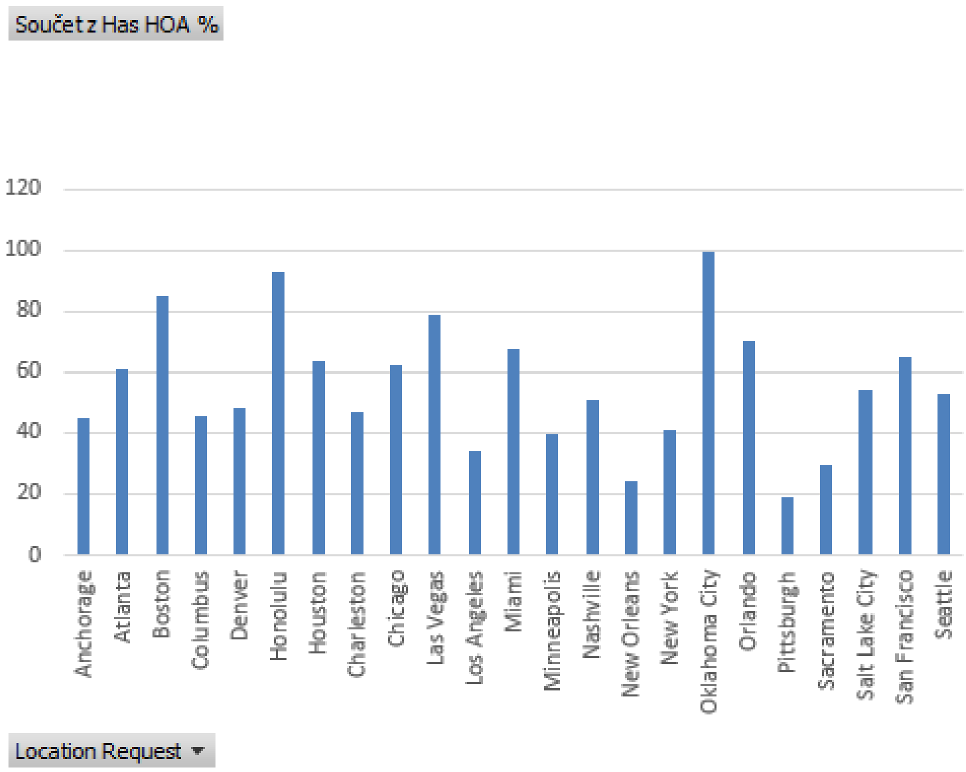

Figure 30 Number of listings containing HOA in % (author)

As we can see on the chart, some locations are more likely to have an HOA. From Oklahoma, with almost 100% of all scrapped listings, to less than 20% in Pittsburgh. One of the highest numbers in the metrics illustrates cities of Miami, Honolulu, San Francisco, and Boston, which, as was already figured out, have a high percentage of condominiums and represent highly urbanized cities. It can be explained that in condominiums, the HOA may own the whole building, and people buying real estate buy part of the building. HOA is responsible for the maintenance of the whole building(repairs, cleaning of the parking and common areas. Also, this type of property more often has conveniences like a gym, a Swimming pool, or a Cinema.

However, the more important metric is the HOA Amount. If HOA can be present, it varies from a few dollars per month for single-family residences to a few thousand dollars for condominiums.

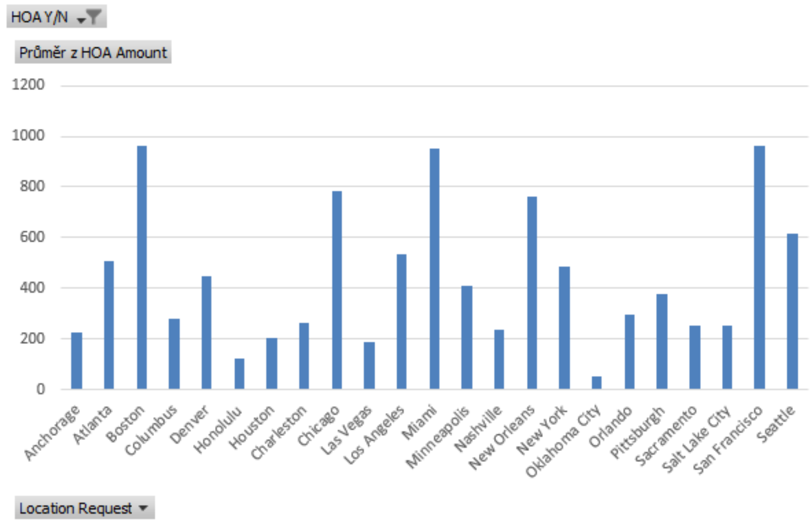

Figure 31 Mean HOA Amount by city (author)

As we can see, the HOA Amount is the highest in 3 areas: San Francisco, Miami, and Boston. In the case of Miami, it may be explained that the city is located in a tourism area, and property objects there might contain more conveniences like pools or access to the pier.

In our dataset, what HOA payment includes is shown in attributes “HOA Includes” and “HOA Features,” which are multi-valued attributes, where certain values are divided by a comma. “HOA Features” contains 288 unique values, while “HOA Includes”- 192.

However, the same values can be described in different ways and may be present both in “HOA Includes” and “HOA Features”.

1. pool	1. water
2. fitness center	2. maintenance grounds
3. clubhouse	3. sewer
4. elevator(s)	4. trash
5. playground	5. insurance
6. storage	6. snow removal
7. spa/hot tub	7. maintenance structure
8. tennis court(s)	8. security
9. gated	9. exterior maintenance
10. exercise room	10. heat
11. sauna	11. pool
12. park	12. gas
13. trash	13. common areas
14. picnic area	14. cable tv
15. patio/deck	15. hot water
16. barbecue	16. internet
17. door person	17. lawn care
18. bbq	18. association management
19. resident manager	19. scavenger
20. pets permitted	20. recreation facilities
21. laundry	21. parking
22. snow removal	22. amenities
23. trash chute	23. common area maintenance
24. security	24. pool(s)
25. wall/fence	25. pest control
26. bike room/bike trails	26. reserve fund
27. business center	27. electricity
28. security guard	28. doorman
29. pool on property	29. air conditioning
30. on site manager/engineer	30. tv/cable

Figure 32 Similar HOA values in HOA Includes and HOA Features (author)

To fix this problem, I have created vocabulary and combined 2 attributes into "HOA Combined". Then, similar values were grouped using vocabulary.

The next code shows how values associated with air conditioning were grouped:

```
cooling_normalization = {
'air conditioning': 'ac',
'a/c': 'ac',
'ac central': 'ac',
'air conditioning allowed': 'ac',
'air conditioning maintenance': 'ac',
}
```

Code 32 Normalizing cooling values (author)

After grouping values, a larger group heatmap can be built, showing the percentage of HOA Services in each location.

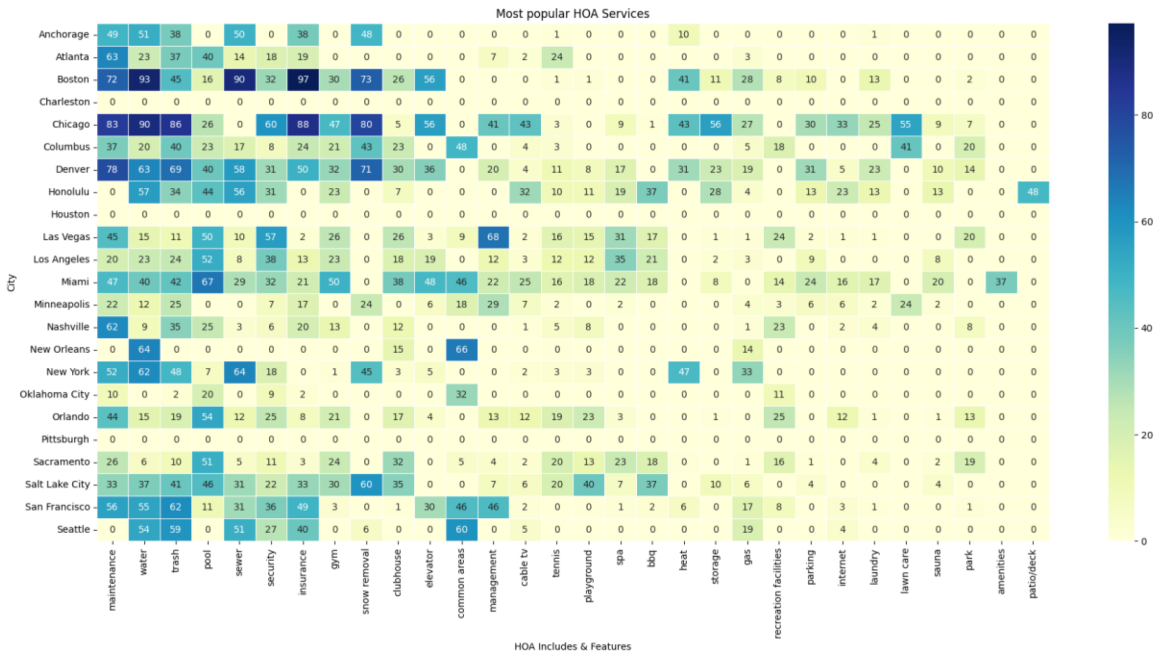

Figure 33 Distribution of Most Common HOA Services by City (author)

Based on the information from the heatmap several key patterns can be emerged from this analysis:

- Core HOA services across cities like maintenance, water, trash, and sewer were present in over 80% of HOA properties in cities like Chicago, Boston, and Denver. These components form the foundation of HOA structures, particularly in dense metropolitan housing environments.
- Amenities like a gym, spa, and recreation facilities are less common but indicate upscale developments. Chicago has 47% of HOA properties, including a gym. Miami includes recreation facilities in 37% of cases. Salt Lake City features spa access in 40% of properties.

- Security Services are concentrated in major urban centers. Cities such as New York (64%), Chicago (60%), Las Vegas (57%), and Denver (50%) show high levels of security inclusion, including doormen, gated entry, and patrols.

Cooling and Heating

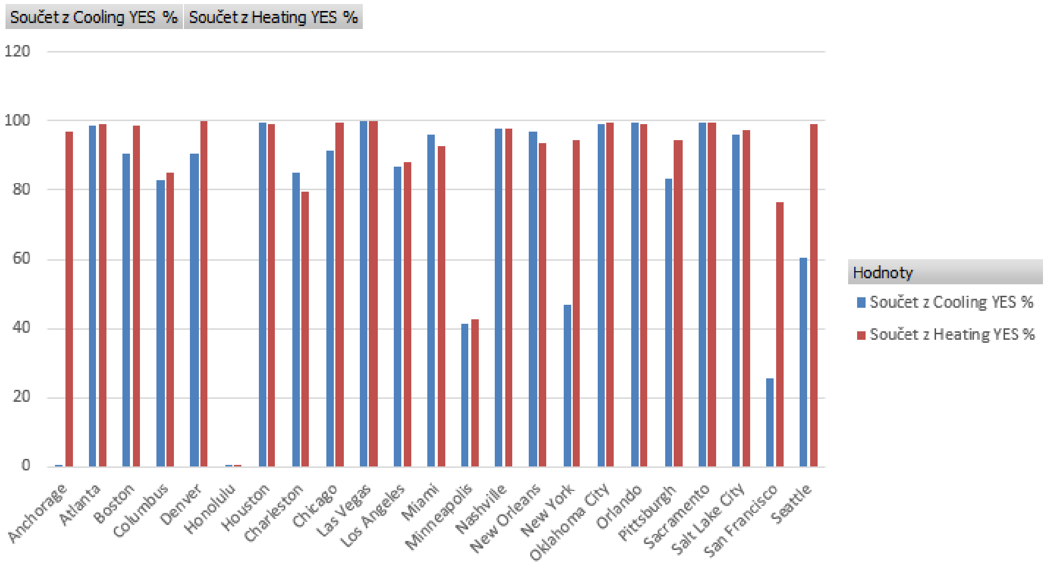

Figure 34 Percentage of listings which contain Cooling/Heating in % (author)

The bar chart X shows in percents how many property listings in each location have installed cooling or heating systems. As we can see, next tendencies can be noted:

- In southern markets such as Miami, Las Vegas, and Orlando, cooling coverage reaches nearly 100%, reflecting hot climates and lifestyle demands. Interestingly, heating is also present in the vast majority of properties. One of the reasons may be construction standards, which require the presence of a cooling system.
- Cities such as New York, San Francisco, and Seattle form a distinct group characterized by universal heating coverage and non-universal cooling adoption. This pattern aligns with their temperate or coastal climates, where winters necessitate reliable heating systems, while summers may not require mechanical cooling in all properties.
- In contrast, northern cities like Chicago and Boston show near-universal heating coverage, while cooling percentages range from 80–90%, indicating a shift toward modern comfort standards but not universal necessity.
- Among outliers can be noted cities like Anchorage, where only less than 1% of listings have cooling, and the city of Honolulu, where heating and cooling are present only in less than 1% of listings, which might be caused by failure to provide data from listing owners. Also, Minneapolis shows only ~40% coverage for both cooling and heating, which is a low figure given its cold climate.

  To show what heating and cooling types are the most popular among selected cities, heatmaps were built.

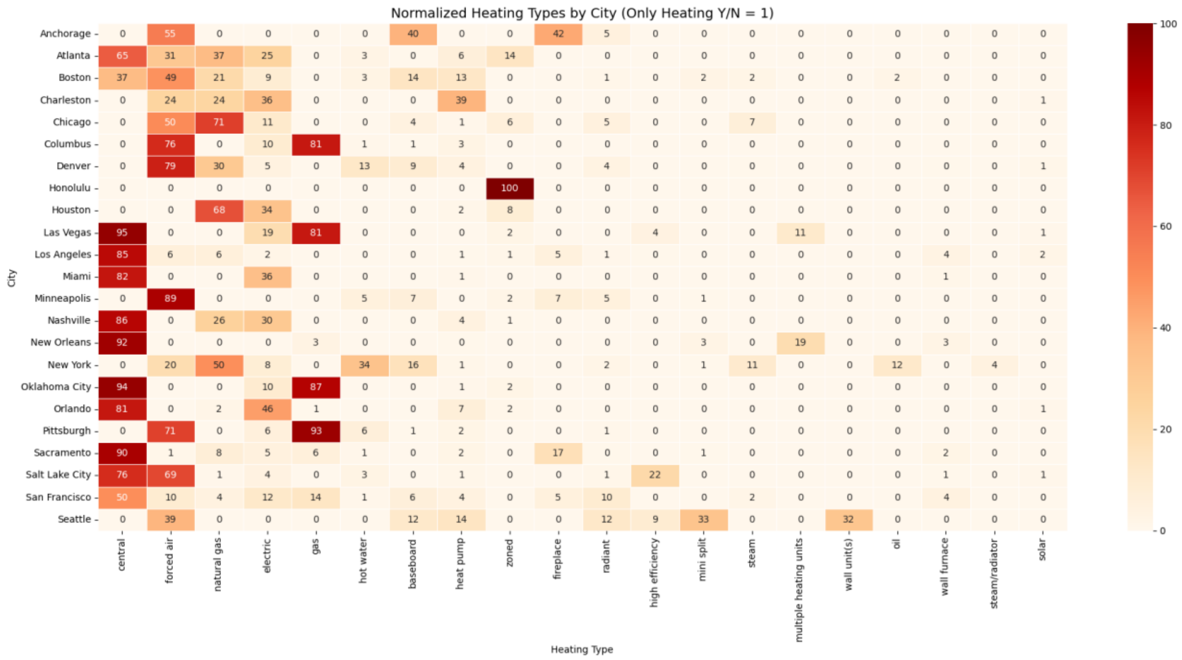

Figure 35 Heating Type heatmap (author)

From the heatmap of the attribute "Heating Type" we can make several conclusions:

- Central heating is the most popular type of heating in warm cities, regardless of the region. The majority of listings are in Las Vegas, Los Angeles, which are closer to the West Coast, and Orlando, New Orleans in the South have this type of heating
- In cold-climate cities like Chicago, Boston, and Columbus, natural gas and forced air systems dominate, indicating reliance on centralized, fuel-based heating.
- Southern markets such as Miami, Orlando, Houston, and Charleston favor electric and heat pump solutions, reflecting both regulatory compliance and mild winter conditions
- Notably, Anchorage diverges from traditional urban setups, with high usage of baseboard, forced air, and even fireplace heating — likely due to decentralized system preferences in remote or rural areas.

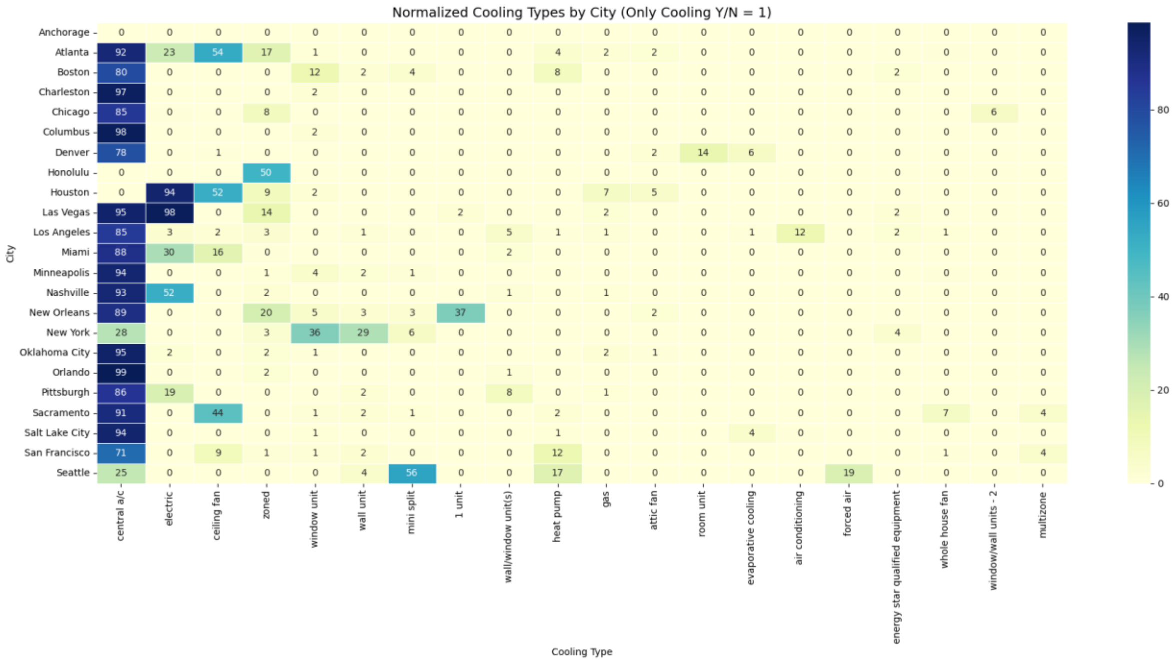

Figure 36 Cooling Type Heatmap (author)

Analyzing the “Cooling Type” heatmap also several conclusion could be made:

- In warm cities such as Miami, Orlando, and Las Vegas, central air dominates, appearing in over 90% of properties.
- Legacy solutions such as wall unit(s) and window unit(s) remain prevalent in New York, where ductwork retrofitting might be costly or impractical.
- The emergence of ductless mini split systems in markets like Seattle indicates a shift toward more energy-efficient and renovation-friendly cooling alternatives.

Interior and Exterior Features

To find the most popular Interior and Exterior Features heatmap will be build as it was done for HOA and Cooling/Heating Type in the previous paragraphs.

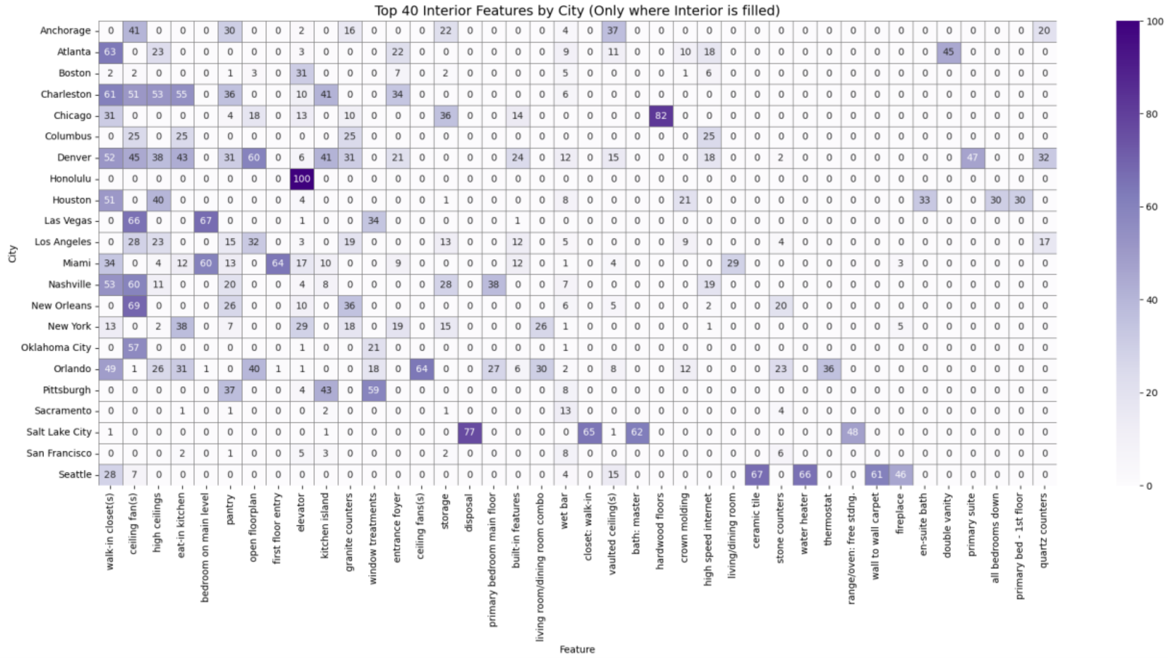

Figure 37 Interior Features Heatmap (author)

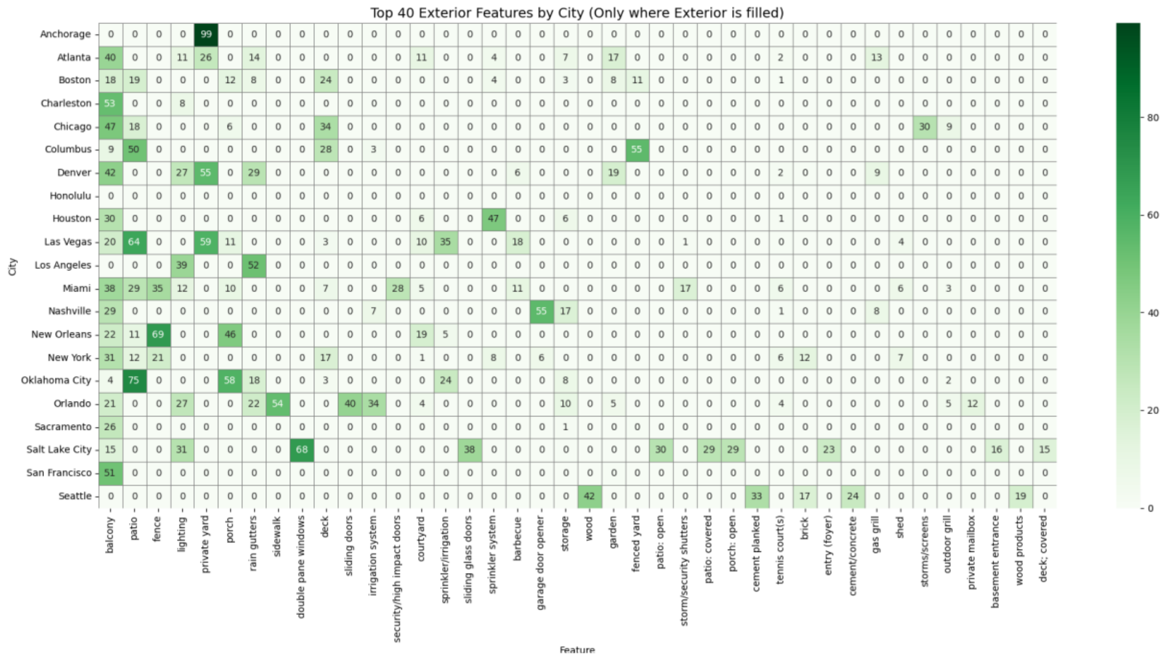

Figure 38 Exterior Features Heatmap (author)

Cities like Miami, Chicago, and San-Francisco report high rates of balconies, a feature associated with condominiums and vertical housing. In contrast, Anchorage, Denver, and Las Vegas show strong representation of private yards, patios, and fenced areas, aligning with suburban planning and single-family homes.

Boston and San Francisco lean toward decks, storm windows, and energy-efficient details, which match their dense historical cores and stricter building codes.

Price/Sq ft

To visualize correlation between price/sq ft and other attributes heatmaps were built.

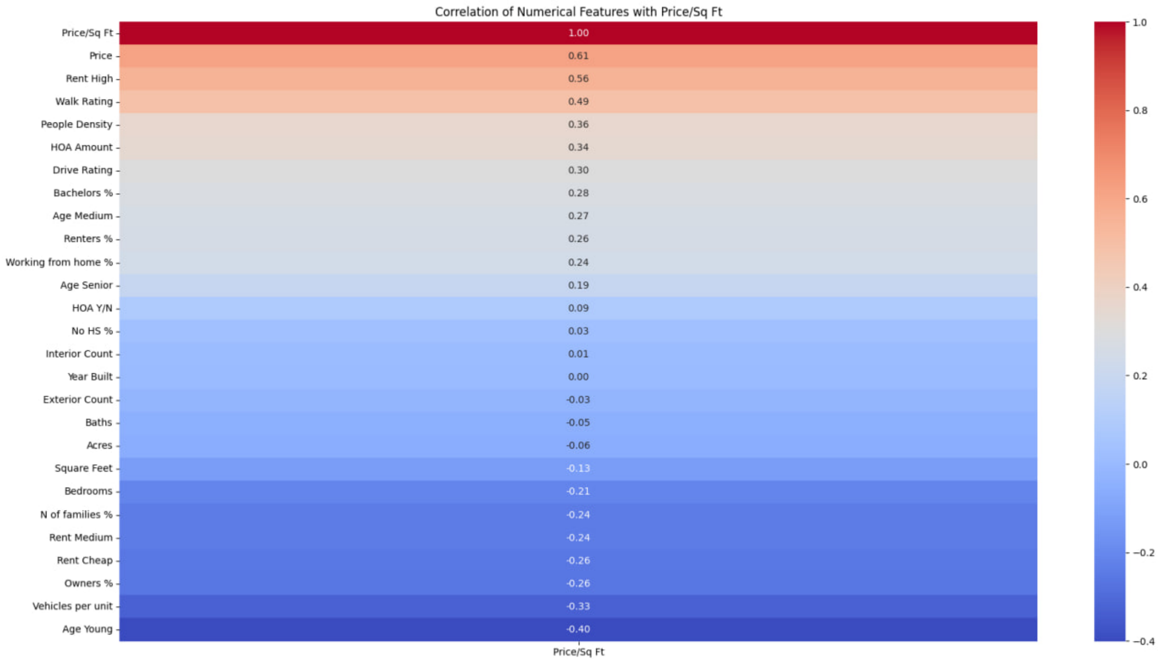

Figure 39 Correlation of numeric values with price/sq ft (author)

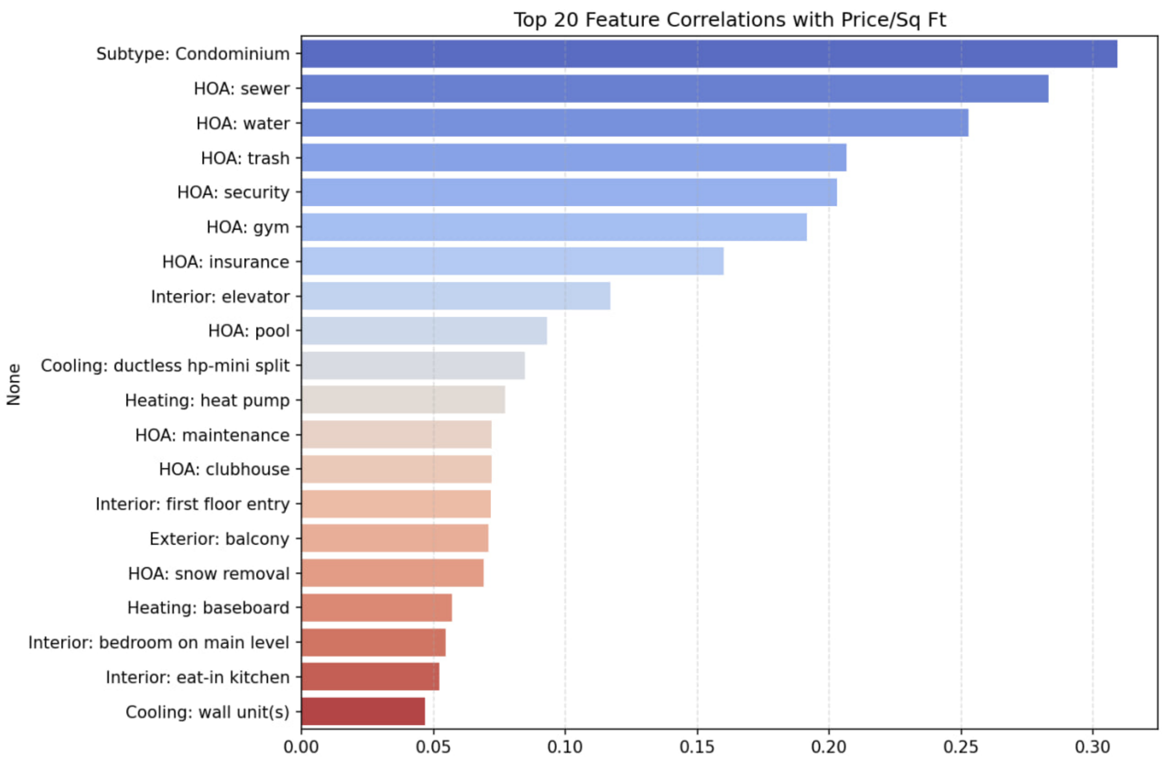

Figure 40 Correlation of selected features with price/sq ft (author)

3.6 Modelling

3.6.1 Clustering

Preparation

In order to uncover structural patterns within the U.S. housing market, an unsupervised clustering analysis will be conducted. The goal is to examine whether cities can be grouped based on similarities in their real estate characteristics, pricing structure, socio-demographic composition, and infrastructure indicators.

Clustering enables the discovery of complex, multidimensional relationships that drive market segmentation. It allows for the identification of city profiles that share similar combinations of price levels, demographic structure, housing types, and lifestyle factors, thus contributing to a deeper understanding of the logic behind price formation and regional housing dynamics in the United States.

During the analysis of demographic attributes I have found out mistake, which was made at the previous step of Web-Scrapping. Attributes "Yes HS %", "No HS %" and "Bachelors %" were presented in absolute values, showing number of people above 25 year old. However, to have them in dataset in percentage value they were divided by value "People Total", which combines all ages. To fix this problem I have decided to calculate number initial absolute value groups for each education group and then divide by total number of people, which are older than 25 years old.

However, to avoid large number of age attributes in the dataset they were combined to bigger groups, what happened with age groups for people between 20 to 24 years old and 25 to 34 years old.

```
ppl_btwn_20_34yo_pct = (population_data_api.get('ppl_btwn_20_24yo') +
population_data_api.get('ppl_btwn_25_34yo')) / people_total * 100
```

Code 33 Calculating the percentage of population aged 20–34 years (author)

To fix this problem we will assume that number of people above 25 years old is 2/3 of total number of people in the age group of 20 to 34 years old and calculate total number of people above 25yo:

```
=(2/3*[@[Age 20_34yo %]]+[@[Age 35_44yo %]]+[@[Age 45_64yo %]]+[@[Age 65_84yo %]]+[@[Age
over 84yo %]])*[@[People Total]]/100
```

Code 34 # Estimating the number of adults aged 25 and older

After this we will calculate absolute numbers of people with and without High School Diploma and Bachelors.

```
=[@[No HS %]]*[@[People Total]]/100
```

```
=[@[Yes HS %]]*[@[People Total]]/100
=[@[Bachelors %]]*[@[People Total]]/100
```
Code 35 Estimating absolute population counts by education level (author)

Then relative number of people in each educational group can be calculated

```
=[@[No HS absolut]]/[@[Age over 25 absolut]]*100
=[@[Yes HS absolut]]/[@[Age over 25 absolut]]*100
=[@[Bachelors absolut]]/[@[Age over 25 absolut]]*100
```
Code 36 # Education level percentages among 25+ population (author)

To reduce dimensional noise and enhance model robustness some age and rent attributes were merged in wider groups: “Rent Cheap” for rent below $800 “Rent Medium” for rent between $800 and $1999 and Rent High for rent over $2000; and Age Young for age group below under 19yo, Age Medium for age group between 20yo and 64yo and Age Senior above 64yo.

Another fixing operation was removing listings with zip-code, which got values of “0” almost in all demographic fields or had very low number of people density(6). Total number of such values is below 10.

Attribute “Property Sub Type” was encoded to be able to use during the clustering.

Methodology

To segment the selected U.S. cities based on similarities in housing market characteristics, demographic structure, and infrastructure quality, hierarchical clustering was conducted using the Agglomerative Clustering algorithm. First, a set of relevant features was selected, including pricing indicators (such as Price and Square Feet), demographic attributes (such as education levels, age composition, rent brackets, and population density), infrastructure ratings (Walk Rating and Drive Rating), and housing-specific variables (HOA Amount and Property Sub Type).

```
features =   [
    'Price', "Square Feet", "Year Built",
    "Drive Rating", "Walk Rating", "N of families %", "People Density",
    "White collar jobs %",
    "HOA Amount",
    "Owners %",
    "Age Senior", "Age Young",
    "Rent High", "Rent Cheap",
    "No HS %", "Bachelors %",
    "White pop %",
    "Property Sub Type_Condominium",
    "Property Sub Type_Single Family Residence"
]
```
Code 37 Selected features for modeling property pricing (author)

These features were aggregated at the city level using mean values, producing a structured dataset representing the average profile of each city. Prior to clustering, the data was standardized using z-score normalization to ensure equal weighting of features regardless of scale. The Agglomerative Clustering algorithm was then applied with a specified number of 4 clusters. Although the silhouette score peaked at k = 3, the difference between scores at k = 3, the 4-cluster solution was ultimately selected because it provided better interpretability and allowed for the identification of more nuanced patterns across city groups.

```
df_grouped = df.groupby("Location Request")[features].mean().reset_index()
df_grouped = df_grouped.sort_values("Location Request").reset_index(drop=True)

#Standardizing features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_grouped[features])
#Agglomerative clustering
model = AgglomerativeClustering(n_clusters=4)
labels = model.fit_predict(X_scaled)
```

Code 38 # Clustering cities based on average feature values (author)

To visualize the results, Principal Component Analysis (PCA) was used to project the high-dimensional feature space onto two principal components, capturing the most significant variance in the data. Each city was plotted in this two-dimensional space, labeled accordingly, and color-coded by cluster assignment. Convex hulls were drawn around the cities within each cluster to illustrate their boundaries and spatial separation.

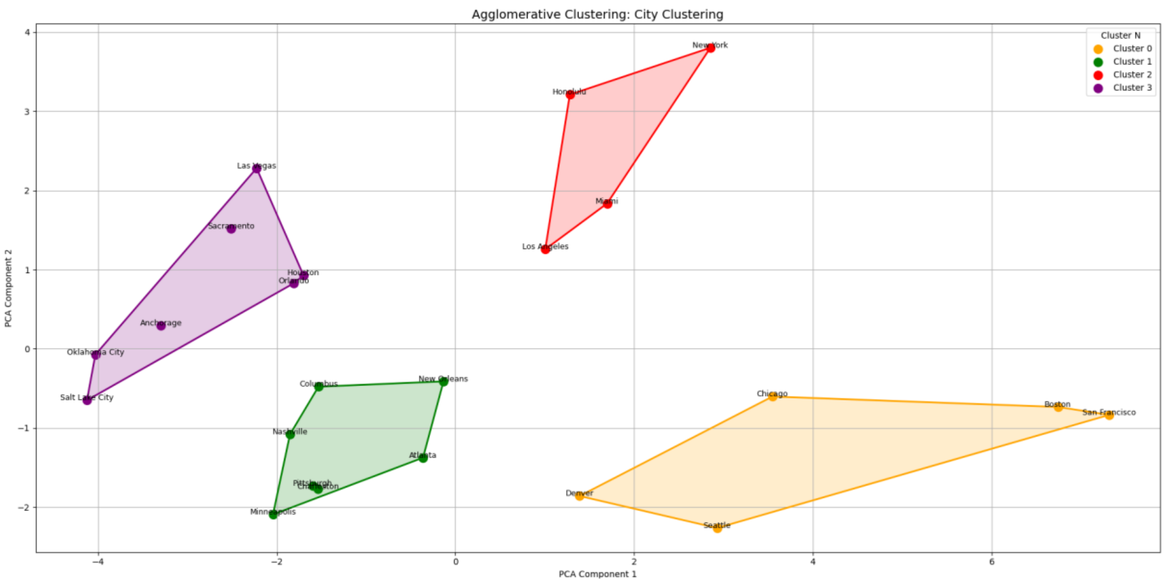

Figure 41 Clusters (author)

3.6.2 Regression modelling

To figure out how certain numeric and categorical attributes influence residential pricing, a supervised learning method, Random Forest regression, was used. While EDA and clusters reveal general patterns across the U.S. housing market, regression modelling was used to evaluate how different variables contribute to price/sq ft.

Random Forest was selected due to its robustness against multicollinearity, ability to handle both categorical and numerical features, and suitability for datasets with nonlinear interactions.

In the global model (across all cities), predictors included numerical characteristics such as Bedrooms, Baths, Year Built, HOA Amount, Walk Rating, Bachelors% , and others, as well as the top-30 most frequent categories for heating/cooling types, HOA features, and property types.

This model was then extended by training separate Random Forest regressors for each city to analyze how the importance of different features varies regionally, supporting the broader research goal of comparing pricing logic across markets.

Modelling Random Forest for all cities in the dataset

To include the 30 most frequent categories from multi-valued categorical attributes next code was used:

```
def top_n_dummies(series, prefix, top_n=30):
    all_items = series.dropna().astype(str).str.split(", ").explode()
    top_categories = all_items.value_counts().head(top_n).index.tolist()

    def filter_top(entry):
        if pd.isna(entry):
            return []
        items = [i.strip() for i in str(entry).split(",")]
        return [i for i in items if i in top_categories]

    filtered = series.apply(filter_top)
    dummies = pd.get_dummies(filtered.explode()).groupby(level=0).max()
    return dummies.add_prefix(prefix + "_")

# One hot encoding for top 15
cooling_dummies = top_n_dummies(df["Cooling Type Normalized"], "cooling", top_n=30)
heating_dummies = top_n_dummies(df["Heating Type Normalized"], "heating", top_n=30)
```
Code 39 Selecting top-30 values from certain attributes (author)

Then, numeric values must be selected:

```
# Choosing numeric values
numeric_features = [
'Baths', "Square Feet", "Year Built",
"Drive Rating", "Walk Rating", "Vehicles per unit", "N of families %", "People Density",
"Cooling Y/N", "Heating Y/N", "HOA Y/N",
"Owners %", "Renters %",
"Age Senior", "Age Young", "Age Medium",
"Rent High", "Rent Cheap", "Rent Medium",
"No HS %", "Yes HS %", "Bachelors %",
"Working from home %", "White collar jobs %", "Blue collar jobs %"


numeric_df = df[numeric_features].copy()
```
Code 40 Selecting numeric values for the model (author)

As a target attribute 'price/sq ft' was selected

```
X = pd.concat([
    cooling_dummies,
    heating_dummies,
    hoa_dummies,
    property_dummies,
    numeric_df
], axis=1)

y = df["Price/Sq Ft"]
```
Code 41 Defining X and Y for Random Forest (author)

The dataset was randomly split into training – 80% and testing – 20% to enable unbiased model evaluation and model will be trained. Number of trees for Random Forest was set on 100.9

```
X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```
Code 42 Training model (author)

Modelling Random Forest for each city

To model random forest for each city separately and get results what attributes influences them the most cycle was built.

```
for city in df["Location Request"].unique():
X_city = pd.concat([
    cooling_dummies_all[city_mask],
    heating_dummies_all[city_mask],
    hoa_dummies_all[city_mask],
```

```
    property_dummies_all[city_mask],
    numeric_df_all[city_mask]
], axis=1)
y_city = df.loc[city_mask, "Price/Sq Ft"]
X_train, X_test, y_train, y_test = train_test_split(X_city, y_city, test_size=0.2,
random_state=42)
model = RandomForestRegressor(...)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```
Code 43 Random forest for each city (author)

3.7 Evaluation

Clustering

Received clusters exhibit a distinct market profile:

- Cluster 0 – Affluent Urban Centers with High Density and Condo Dominance: San Francisco, Boston, Chicago

  This cluster represents some of the most economically advanced and densely populated cities. It has the highest average property price ($1,042,302) despite having smaller average home sizes (1,612 sq ft). Cities in this group score very high in white-collar job concentration (78.3%), Walk Rating (3.91), and Drive Rating (4.69), suggesting mature infrastructure and strong economic activity.

  They also exhibit the highest HOA fees ($495) and a notable presence of condominiums (53%), with only 38% being single-family residences. Educational attainment is also highest here, with 34% of the population over 25 holding a Bachelor's degree, while the share of residents without a high school diploma is relatively low (5.6%). These cities are typically diverse (64.7% white population) and have a relatively younger age structure.

- Cluster 1 – Spacious Suburban Markets with Family-Oriented Demographics: Atlanta, Pittsburgh, Minneapolis, Nashville, Columbus.

  This group is characterized by larger average property sizes (2,076 sq ft) and moderate prices ($624,426). It shows strong ownership rates (56%), a high share of single-family homes (78%), and low condo presence (12%). Demographically, this cluster skews toward families and older residents (Age Senior: 13.2%), with the lowest share of renters paying high rent (6%).

  The educational profile is above average, with 27% holding a Bachelor's degree, and only 3% without high school education. The cities here have low to moderate density (3,300 ppl/sq mi), lower HOA fees ($156), and less walkable environments (Walk Rating: 2.49).

- Cluster 2: High-Density Mixed Markets with Rising Costs and Demographic Complexity: New York, Los Angeles, Miami, Honolulu.

  Cluster 2 combines cities with high population density (12,031 ppl/sq mi), elevated property prices ($963,537), and a higher proportion of renters paying expensive rents (22%). These cities are diverse and urbanized, though with lower educational attainment (only 23% with a Bachelor's degree) and higher share without HS diploma (7.2%).

  Ownership is lower (51%) and condos are relatively common (43%), indicating a multifamily rental housing structure. Infrastructure indicators are strong (Drive: 4.38, Walk: 3.16), and the share of white population is lowest (52.5%), indicating demographic diversity.

- Cluster 3 – Affordable Emerging Cities with Younger Populations and Low HOA Burden : Salt Lake City, Oklahoma City, Houston, Orlando, Anchorage, Las Vegas.

  This cluster groups cities with the lowest average home price ($554,606) and the largest average home size (2,082 sq ft). It is marked by high homeownership (60%), family-centric demographics, and a young population profile (Age Young: 27%).

  Cities in this cluster are also characterized by low HOA fees ($120), dominance of single-family housing (78%), and lower walkability (2.08). Educational levels are relatively low, with 22% having a Bachelor's degree and 5% without a high school diploma. The low density (3,127 ppl/sq mi) and modest infrastructure scores suggest suburban or developing markets.

Random Forest

The global model, trained on the full dataset, achieved an R^2 of 0.790, a mean absolute error (MAE) of $71.09/sq ft, and a root mean squared error (RMSE) of $130.78/sq ft, indicating strong overall performance.

Among the features that influence price/sq feet the most were selected next ones:

Feature	Importance
Rent High	0.301075
Walk Rating	0.138920
Year Built	0.100235
Square feet	0.081601
Drive Rating	0.026201
Vehicles per unit	0.019075
Baths	0.017232
Age Young	0.016524
Working from home %	0.015713
Age Senior%	0.013579

Table 12 The most important features in the global model (author)

Since all the most important features are numeric attributes, I have decided to build a model, excluding them, only with categorical attributes. The model explains approximately 33.6% of the variance in price per square foot, which can still be considered a meaningful result. Predictions deviate from actual values by about $151, and the root mean squared error is $232.73/sq ft.

Feature	**Importance**
prop_type_Condominium	0.196
heating_gas	0.073
heating_central	0.053
hoa_security	0.048
hoa_water	0.034

Table 13 The most important features among categorial values (author)

To explore regional differences in price formation logic, individual Random Forest regression models were trained for each city in the dataset. The goal was to assess not only the predictive power of the models within each urban market but also to compare which features play the most important roles in determining the price per square foot (Price/Sq Ft) locally.

City	R^2	MAE	RMS	Top Features
Seattle	0,601	101,74	170,14	Square Feet (0.236); Year Built (0.166); hoa_security (0.053); Baths (0.041); Blue collar jobs % (0.038)
Pittsburgh	0,234	52,62	141,22	Square Feet (0.216); Year Built (0.138); Yes HS % (0.120); No HS % (0.088); Baths (0.050)
Anchorage	0,54	38,2	53,31	Year Built (0.349); Square Feet (0.207); Baths (0.056); Bachelors % (0.050); Age Senior (0.026)
San Franci	0,241	181,58	261,26	Square Feet (0.215); Year Built (0.167); Working from home % (0.062); Yes HS % (0.061); No HS % (0.049)
Houston	0,681	37,11	66,62	Square Feet (0.168); Year Built (0.143); White collar jobs % (0.113); Blue collar jobs % (0.097); Rent High (0.061)
Nashville	0,677	64,23	107,66	Square Feet (0.171); Year Built (0.148); Walk Rating (0.123); N of families % (0.099); Rent High (0.057)
Atlanta	0,769	36,73	56,9	Year Built (0.167); Square Feet (0.118); Yes HS % (0.114); Age Young (0.107); Rent High (0.067)
Honolulu	0,641	133,58	209,05	Year Built (0.340); Square Feet (0.236); Age Young (0.073); N of families % (0.060); Bachelors % (0.048)
Chicago	0,739	53,99	78,81	Rent High (0.311); Square Feet (0.150); Year Built (0.117); Working from home % (0.061); heating_heat pump (0.031)
Oklahoma	0,494	22,52	40,78	Year Built (0.229); Square Feet (0.222); Age Medium (0.100); Age Senior (0.075); N of families % (0.062)
Boston	0,777	160,3	228,03	Walk Rating (0.374); Year Built (0.163); Square Feet (0.082); Age Medium (0.043); Yes HS % (0.039)
New York	0,661	95,57	160,94	Year Built (0.286); Square Feet (0.121); Blue collar jobs % (0.061); Baths (0.057); White collar jobs % (0.056)
Los Angel	0,668	108,1	158,71	Yes HS % (0.189); Year Built (0.128); Square Feet (0.123); Drive Rating (0.106); Working from home % (0.079)
Minneapc	0,504	38,63	63,76	Year Built (0.205); Square Feet (0.159); Bachelors % (0.155); Baths (0.076); Age Young (0.064)
Sacramen	0,38	41,69	68,8	Square Feet (0.205); Year Built (0.114); Walk Rating (0.104); Baths (0.039); Blue collar jobs % (0.038)
Charlestoı	0,719	95,18	145,06	Year Built (0.226); Yes HS % (0.128); Square Feet (0.119); No HS % (0.117); Rent High (0.096)
Columbus	0,479	36,85	55,38	Year Built (0.212); Yes HS % (0.200); Square Feet (0.152); Rent High (0.059); Baths (0.048)
New Orle	0,631	57,27	80,82	Working from home % (0.186); Square Feet (0.171); Year Built (0.131); N of families % (0.091); Bachelors % (0.062)
Las Vegas	0,601	38,07	64,41	Square Feet (0.251); Yes HS % (0.165); Year Built (0.106); cooling_central a/c (0.069); Bachelors % (0.052)
Salt Lake (	0,598	36,54	58,04	Age Young (0.248); Square Feet (0.219); Year Built (0.121); Baths (0.034); heating_baseboard (0.034)
Miami	0,539	119,79	201,94	N of families % (0.178); Year Built (0.168); Square Feet (0.168); Baths (0.065); Walk Rating (0.033)
Orlando	0,426	37,91	72,72	Square Feet (0.202); Rent High (0.199); Year Built (0.171); No HS % (0.040); Baths (0.039)
Denver	0,652	60,99	86,7	Walk Rating (0.257); Square Feet (0.190); Year Built (0.167); Bachelors % (0.028); Baths (0.027)

Table 14 Random Forest results for each city(author)

The table shows that model performance varies significantly across cities: the best results (highest R^2 and lowest MAE/RMSE) were observed in Charleston, Houston, and Columbus, where housing markets appear more predictable. In contrast, cities like San Francisco and Sacramento showed poor model accuracy.

Conclusion

The aim of this bachelor's thesis was to collect data on the residential real estate market in the United States through web scraping and to analyze it using exploratory data analysis and data mining techniques in order to identify patterns, relationships between attributes, and regional differences within the market. To accomplish this, the project was divided into several stages. First, a large dataset of residential listings was collected using web scraping from a major U.S. real estate platform and cleaned to ensure consistency and analytical value. In the exploratory data analysis phase, we investigated structural differences between cities in terms of property types, size, pricing, demographics, infrastructure (e.g., walkability and driveability), and the presence and nature of HOA services. Next, clustering techniques were applied to group cities based on aggregated characteristics, revealing distinct market profiles and underlying segmentation in the U.S. housing landscape. Finally, regression models—both a global Random Forest model and individual models for each city—were trained to evaluate how specific features impact pricing behavior and to compare the importance of variables across urban markets.

The main contribution of this work lies in its integrated approach to combining real estate data with socio-demographic and infrastructure variables, producing a comprehensive analysis of regional housing dynamics. The findings provide valuable insights for real estate investors, researchers, and policymakers by highlighting how different factors—ranging from property features to education levels and walkability—interact within local markets. Possible extensions of this research include incorporating time-series data to observe market trends over time, expanding the analysis to rental markets, or experimenting with alternative machine learning algorithms such as XGBoost or neural networks to improve predictive accuracy and deepen insight into complex market behavior.

List of references

BeautifulSoup Web Scraping: Step-By-Step Tutorial. (n.d.). Bright Data. Retrieved November 10, 2024, from https://brightdata.com/blog/how-tos/beautiful-soup-web-scraping

Bhattacharyya, S., & Timilsina, G. (2009). *Energy Demand Models for Policy Formulation: A Comparative Study of Energy Demand Models*.

Broucke, S. vanden, & Baesens, B. (2018). *Practical Web Scraping for Data Science: Best Practices and Examples with Python* (1st ed. edition). Apress.

Bureau, U. C. (n.d.). *Decennial Census of Population and Housing*. Census.Gov. Retrieved May 10, 2025, from https://www.census.gov/programs-surveys/decennial-census.html

Chapman, A. D. (2005). *Principles and methods of data cleaning*. GBIF.

Crews, K. D. (2006). *Copyright Law for Librarians and Educators: Creative Strategies and Practical Solutions*. American Library Association.

Dong, X. L., & Srivastava, D. (2022). *Big Data Integration*. Springer Nature.

Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From Data Mining to Knowledge Discovery in Databases. *AI Magazine*, *17*(3), Article 3. https://doi.org/10.1609/aimag.v17i3.1230

Frawley, W. J., Piatetsky-Shapiro, G., & J. Matheus, C. (1992). *Knowledge Discovery in Databases: An Overview*.

Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques*. Elsevier.

History of the United States. (2024). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=History_of_the_United_States&oldid=1216621776

Li, R. (n.d.). *History of Data Mining*. KDnuggets. Retrieved February 4, 2024, from https://www.kdnuggets.com/history-of-data-mining

Malik, A. (n.d.). *Subject and Course Guides: Digital UIC: Static and Dynamic Websites*. Retrieved November 11, 2024, from https://researchguides.uic.edu/digital-uic/static-dynamic

Mariotti, T. (2024, January 13). *Top 10 Real Estate Websites in the U.S. (2024)*. RubyHome.Com. https://www.rubyhome.com/blog/most-visited-real-estate-websites/

Mitchell, R. (2018). *Web Scraping with Python: Collecting More Data from the Modern Web* (2nd edition). O'Reilly Media.

Mobile home. (2025). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Mobile_home&oldid=1275892513

Pegarkov, D. D. (2006). *National Security Issues*. Nova Publishers.

Piatetsky-Shapiro, G. (2000). Knowledge discovery in databases: 10 years after. *ACM SIGKDD Explorations Newsletter*, *1*(2), Article 2. https://doi.org/10.1145/846183.846197

Real Estate—United States | Statista Market Forecast. (n.d.). Statista. Retrieved April 1, 2024, from https://www.statista.com/outlook/fmo/real-estate/united-states

Scrapfly Web Scraping API | Academy—Scraping Static HTML. (n.d.). Retrieved November 10, 2024, from https://scrapfly.io/academy/static-scraping

Scrapy at a glance—Scrapy 2.11.2 documentation. (n.d.). Retrieved November 13, 2024, from https://docs.scrapy.org/en/latest/intro/overview.html

Singhal, N. & Himanshu. (2022). A Review on Knowledge Discovery from Databases. In P. K. Mallick, A. K. Bhoi, A. González-Briones, & P. K. Pattnaik (Eds.), *Electronic Systems and Intelligent Computing* (Vol. 860, pp. 457–464). Springer Nature Singapore. https://doi.org/10.1007/978-981-16-9488-2_43

Swapna, S., Niranjan, P., Srinivas, B., & Swapna, R. (2016). Data cleaning for data quality. *2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*, 344–348. https://ieeexplore.ieee.org/abstract/document/7724284

United States. (2024). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=United_States&oldid=1216452496#cite_note-largestcountry-13

United States Department of Housing and Urban Development. (2024). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=United_States_Department_of_Housing_and_Urban_Development&oldid=1215508558

U.S. Copyright Office Fair Use Index. (n.d.). Retrieved May 10, 2025, from https://www.copyright.gov/fair-use/

US EPA, O. (2013, January 29). *Why Are Our Regional Offices and Labs Located Where They Are? A Historical Perspective on Siting* [Reports and Assessments]. https://www.epa.gov/history/why-are-our-regional-offices-and-labs-located-where-they-are-historical-perspective-siting

Wagh, S. (n.d.). *Research Guides: Public Health Research Guide: Primary & Secondary Data Definitions*. Retrieved February 5, 2024, from https://researchguides.ben.edu/c.php?g=282050&p=4036581

What Is a Headless Browser and Where Is It Used? (n.d.). Retrieved November 12, 2024, from https://oxylabs.io/blog/what-is-headless-browser

Wieringa, J. (2012). Intro to Beautiful Soup. *Programming Historian*, *1*. https://doi.org/10.46430/phen0008

Wirth, R., & Hipp, J. (2000). *CRISP-DM: Towards a Standard Process Model for Data Mining*. https://www.semanticscholar.org/paper/CRISP-DM%3A-Towards-a-Standard-Process-Model-for-Data-Wirth-Hipp/48b9293cfd4297f855867ca278f7069abc6a9c24

Wright, J., Ackley, R., Gopal, S., & Phillips, N. (2022). The BosWash Infrastructure Biome and Energy System Succession. *Infrastructures*, *7*(7), Article 7. https://doi.org/10.3390/infrastructures7070095

Attachments

Attachment A : Datasets

Attachment contain initial dataset and final dataset.

1. all-csvs_excel.xlsx – initial dataset
2. final dataset.xlsx – final dataset used for EDA, Clustering and Regression